5. Szkeleton tervezése

48 – GonoszOnosz

Konzulens:

Marcsingó Ádám

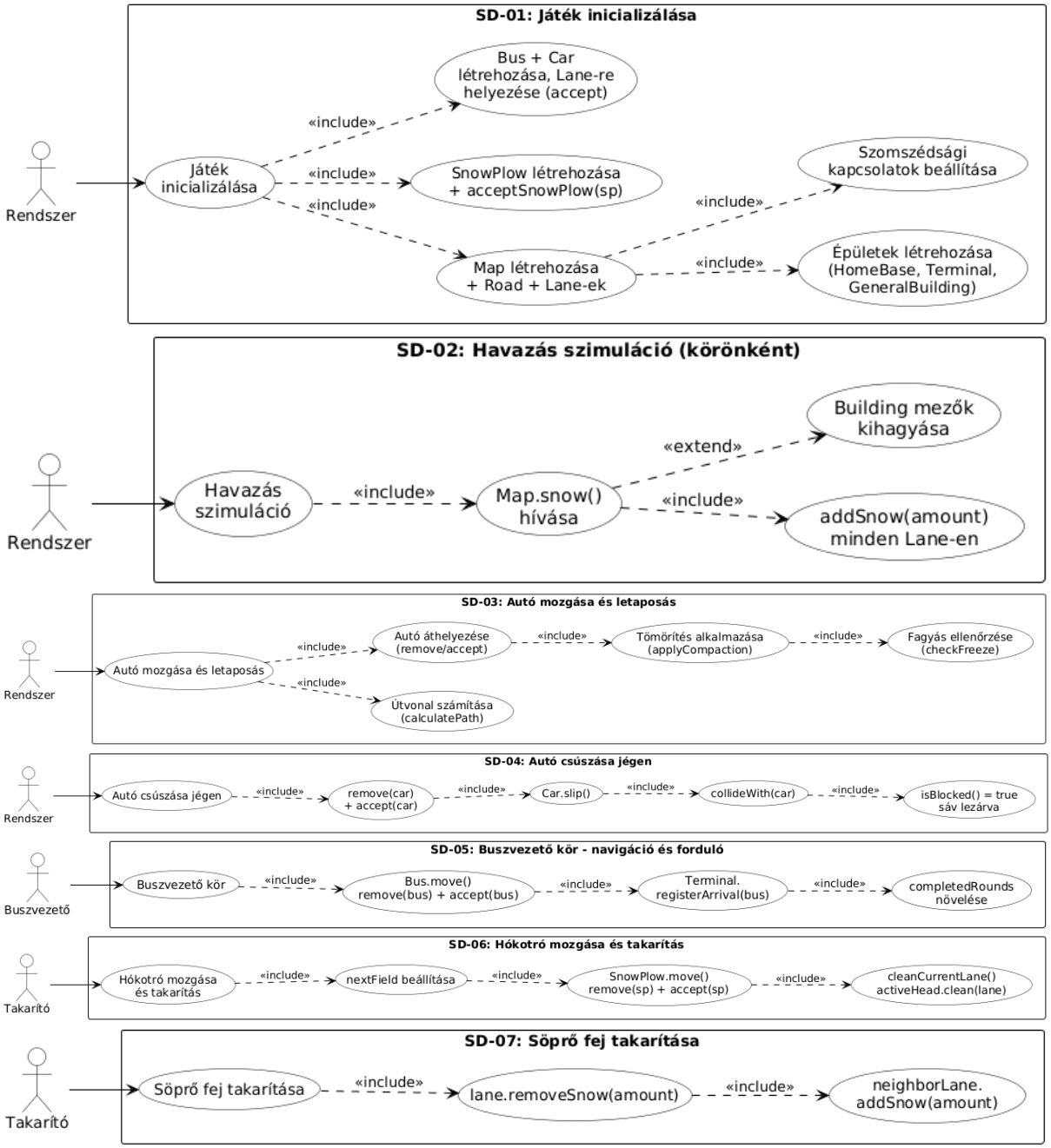
Csapattagok

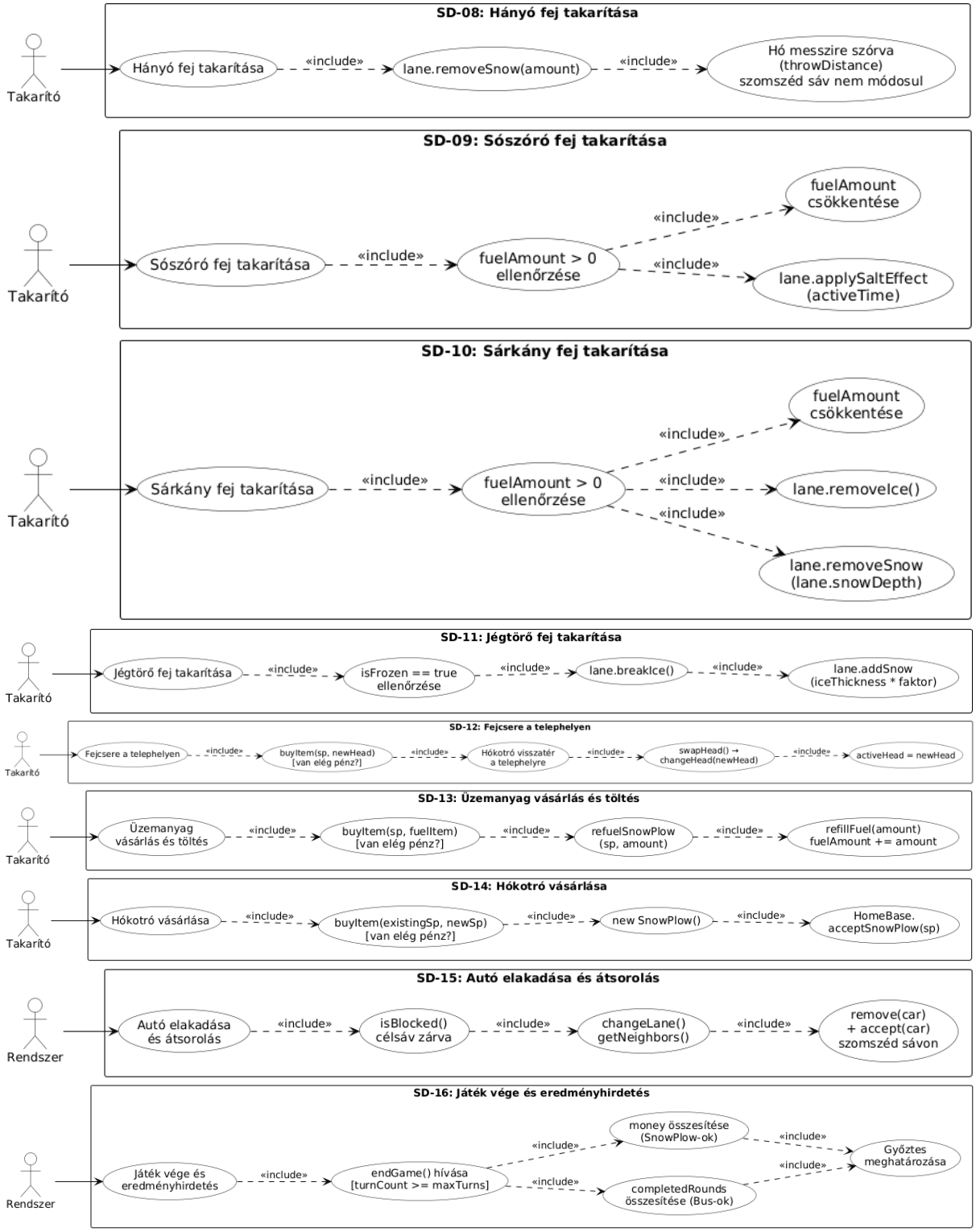
Kálmán Patrik	XG5YQ1	kalmanpatrik1214@gmail.com
Papp Bálint	HND2AH	xypapbal@gmail.com
dr. Grób László Mihály	EAS7U0	laszlo.m.grob@gmail.com
Héja Gergő Zoltán	FEFSRB	hejagergo@gmail.com
Dobronyi Benedek Márk	IMQ1AR	benedek.dobronyi@gmail.com

2026.03.22

6. Szkeleton tervezése

6.1 A szkeleton modell valóságos use-case-ei




SD-08: Hányó fej takarítása
Takarító
Hányó fej takarítása
«include»
lane.removeSnow(amount)
«include»
Hó messzire szórva (throwDistance) szomszéd sáv nem módosul
SD-09: Sószóró fej takarítása
Takarító
Sószóró fej takarítása
«include»
fuelAmount > 0 ellenőrzése
«include»
fuelAmount csökkentése
«include»
lane.applySaltEffect (activeTime)
SD-10: Sárkány fej takarítása
Takarító
Sárkány fej takarítása
«include»
fuelAmount > 0 ellenőrzése
«include»
fuelAmount csökkentése
«include»
lane.removeIce()
«include»
lane.removeSnow (lane.snowDepth)
SD-11: Jégtörő fej takarítása
Takarító
Jégtörő fej takarítása
«include»
isFrozen == true ellenőrzése
«include»
lane.breakIce()
«include»
lane.addSnow (iceThickness * faktor)
SD-12: Fejcsere a telephelyen
Takarító
Fejcsere a telephelyen
«include»
buyItem(sp, newHead) [van elég pénz?]
«include»
Hókotró visszatér a telephelyre
«include»
swapHead() → changeHead(newHead)
«include»
activeHead = newHead
SD-13: Üzemanyag vásárlás és töltés
Takarító
Üzemanyag vásárlás és töltés
«include»
buyItem(sp, fuelItem) [van elég pénz?]
«include»
refuelSnowPlow (sp, amount)
«include»
refillFuel(amount) fuelAmount += amount
SD-14: Hókotró vásárlása
Takarító
Hókotró vásárlása
«include»
buyItem(existingSp, newSp) [van elég pénz?]
«include»
new SnowPlow()
«include»
HomeBase. acceptSnowPlow(sp)
SD-15: Autó elakadása és átsorolás
Rendszer
Autó elakadása és átsorolás
«include»
isBlocked() célsáv zárva
«include»
changeLane() getNeighbors()
«include»
remove(car) + accept(car) szomszéd sávon
SD-16: Játék vége és eredményhirdetés
Rendszer
Játék vége és eredményhirdetés
«include»
endGame() hívása [turnCount >= maxTurns]
«include»
money összesítése (SnowPlow-ok)
«include»
completedRounds összesítése (Bus-ok)
«include»
«include»
Győztes meghatározása

5.1.2 Use-case leírások

Use-case neve	SD- 01: Játék inicializálása
Rövid leírás	A rendszer létrehozza a játék összes objektumát: pályaelemeket, épületeket, járműveket, játékosokat, és elhelyezi őket a gráfban.
Aktorok	Rendszer (automatikus indítás)
Forgatókönyv	1. A GameLogic meghívja a startGame() metódust. 2. Létrejön a Map objektum. 3. A Map létrehoz egy Road-ot, amely létrehozza a Lane objektumokat (l1, l2). 4. A Map létrehozza a HomeBase, Terminal és GeneralBuilding objektumokat. 5. Beállítódnak a szomszédsági kapcsolatok (getNeighbors). 6. Létrejön egy SnowPlow, amelyet a HomeBase acceptSnowPlow() fogad. 7. Létrejön egy Bus, amelyet l1 Lane fogad (accept). 8. Létrejön egy Car, amelyet l1 Lane fogad (accept).

Use-case neve	SD-02: Havazás szimuláció (körönként)
Rövid leírás	Minden kör végrehajtási fázisában a rendszer szimulált havat ad minden Lane típusú mezőhöz.
Aktorok	Rendszer (GameLogic, Map)
Forgatókönyv	1. A GameLogic.advanceTurn() meghívja a Map.snow() metódust. 2. A Map végigiterál a fields listán. 3. Minden Lane típusú mezőn meghívja az addSnow(amount) metódust. 4. A snowDepth értéke növekszik minden érintett sávon. 5. Building és egyéb Field típusokon nem történik hóadás.

Use-case neve	SD-03: Autó mozgása és letaposás
Rövid leírás	Az autó a legrövidebb útvonalon navigál a célépület felé, és áthaladáskor letapossa a havat
Aktorok	Rendszer (GameLogic, Car)
Forgatókönyv	1. A GameLogic.advanceTurn() meghívja a Car.move() metódust. 2. A Car.calculatePath() meghívja a gameMap.findShortestPath(currentLane, target) metódust. 3. A nextField beállítódik a következő Lane-re (pl. l2). 4. Az előző Lane-en remove(car) hívódik. 5. A célLane-en accept(car) hívódik. 6. A célLane-en applyCompaction() hívódik, compactionCount nő. 7A checkFreeze() metódus futásakor a Skeleton megkérdezi a felhasználót: 'Elérte a letaposások száma a küszöböt, és van hó a sávon? (I/N)'. 'I' válasz esetén a program naplózza, hogy a sáv jegessé (isFrozen = true) vált.

Use-case neve	SD-04: Autó csúszása jégen
Rövid leírás	Jégpáncélos sávra lépéskor az autó véletlenszerűen megcsúszik, ütközést okozhat, és lezárja az érintett sávot.
Aktorok	Rendszer (GameLogic, Car)
Forgatókönyv	1. A Car.move() kiszámítja az útvonalat, a nextField = l2 (isFrozen == true). 2. remove(car) hívódik az előző sávon, accept(car) az l2-n. 3. Az l2 sávra lépéskor a Skeleton megkérdezi a felhasználótól: 'Jeges a sáv? (I/N)'. 'I' válasz esetén a Skeleton ismét kérdez: 'Megcsúszik az autó? (I/N)'. 'I' válasz esetén a Car.slip() hívódik.4. A slip() metódusban a Skeleton megkérdezi: 'Van másik jármű a sávban, amivel ütközhet? (I/N)'. 'I' válasz esetén a collideWith(car) hívódik a másik járművön. 5. isBlocked() igazzá válik az érintett sávon. 6. A sáv meghatározott körökig lezárt marad.

Use-case neve	SD-05: Buszvezető kör – navigáció és forduló
Rövid leírás	A buszvezető játékos irányítja a buszt a végállomások között; végállomásra érkezéskor a forduló rögzítésre kerül.
Aktorok	Buszvezető (játékos)
Forgatókönyv	1. A GameLogic.manageTurn() jelzi a buszvezető aktornak a kör kezdetét (startTurn()). 2. A buszvezető kiválasztja a következő célmezőt, meghívja a Bus.move()-t. 3. remove(bus) hívódik az előző Lane-en, accept(bus) a célmezőn. 4. A lépés után a Skeleton megkérdezi: 'A célmező végállomás (Terminal)? (I/N)'. 'I' válasz esetén a Terminal.registerArrival(bus) hívódik. 5. A buszvezető completedRounds értéke nő. 6. A buszvezető meghívja az endTurn()-t.

Use-case neve	SD-06: Hókotró mozgása és takarítás
Rövid leírás	A takarító játékos beállítja a hókotró következő célmezőjét, a hókotró odamegy és a felszerelt fejjel megtisztítja a sávot.
Aktorok	Takarító (játékos)
Forgatókönyv	1. startTurn() jelzi a takarítónak a kör kezdetét. 2. A takarító beállítja: nextField = l2. 3. endTurn() jelzi a döntési fázis végét. 4. A GameLogic.advanceTurn() meghívja a SnowPlow.move()-t. 5. remove(sp) hívódik l1-en, accept(sp) hívódik l2-n. 6. A SnowPlow.cleanCurrentLane() meghívja az activeHead.clean(l2) metódust. 7. A fej elvégzi a sávra jellemző takarítási műveletet.

Use-case neve	SD-07: Söprő fej takarítása
Rövid leírás	A SweepHead eltávolítja a havat a sávról, és áttolják azt a szomszédos sávba.

Aktorok	Takarító (játékos), SnowPlow
Forgatókönyv	1. A SnowPlow.cleanCurrentLane() meghívja a SweepHead.clean(lane) metódust. 2. A SweepHead meghívja lane.removeSnow(amount)-t: a hóréteg csökken. 3. lane.getNeighbors() visszaadja a szomszédos sávokat. 4. A szomszéd neighborLane-en addSnow(amount) hívódik. 5. A hó átkerül a szomszédos sávba.

Use-case neve	SD-08: Hányó fej takarítása
Rövid leírás	A ThrowerHead eltávolítja a havat a sávról, és messzire (pl. a járdára) szórja, szomszédos sávot nem módosít.
Aktorok	Takarító (játékos), SnowPlow
Forgatókönyv	1. A SnowPlow.cleanCurrentLane() meghívja a ThrowerHead.clean(lane) metódust. 2. A ThrowerHead meghívja lane.removeSnow(amount)-t. 3. A throwDistance értéknek megfelelően a hó messzire kerül (járdára). 4. Szomszédos Lane-en nem történik addSnow() hívás.

Use-case neve	SD-09: Sószóró fej takarítása
Rövid leírás	A SaltHead sót szór a sávra, amely fokozatosan olvasztja a havat és jeget, és megakadályozza az új hó megmaradását; üres tartály esetén nincs hatás.
Aktorok	Takarító (játékos), SnowPlow
Forgatókönyv	1. A SnowPlow.cleanCurrentLane() meghívja a SaltHead.clean(lane)-t. A clean(lane) hívásakor a Skeleton megkérdezi: 'Van elegendő só a tartályban (fuelAmount > 0)? (I/N)'. 'I' válasz esetén a lane.applySaltEffect(activeTime) hívódik. 'N' válasz esetén a metódus visszatér anélkül, hogy bármit csinálna.

Use-case neve	SD-10: Sárkány fej takarítása
Rövid leírás	A DragonHead biokerozin elégetésével azonnal eltávolít minden havat és jeget a sávról; üres tartály esetén nincs hatás.
Aktorok	Takarító (játékos), SnowPlow
Forgatókönyv	1. A SnowPlow.cleanCurrentLane() meghívja a DragonHead.clean(lane)-t. 2A clean(lane) hívásakor a Skeleton megkérdezi: 'Van elegendő biokerozin a tartályban (fuelAmount > 0)? (I/N)'. 'I' válasz esetén lefutnak a removeSnow() és removeIce() hívások. 'N' válasz esetén a metódus visszatér.

Use-case neve	SD-11: Jégtörő fej takarítása
Rövid leírás	Az IcebreakerHead feltöri a jégpáncélt és hóvá alakítja; ha nincs jég, nem csinál semmit.
Aktorok	Takarító (játékos), SnowPlow
Forgatókönyv	1. A SnowPlow.cleanCurrentLane() meghívja az IcebreakerHead.clean(lane)-t. 2. A clean(lane) hívásakor a Skeleton megkérdezi: 'Jeges a sáv (isFrozen == true)? (I/N)'. 'I' válasz esetén lefut a breakIce() és addSnow(). 'N' válasz esetén a metódus visszatér.

Use-case neve	SD-12: Fejcsere a telephelyen
Rövid leírás	A takarító játékos új fejet vásárol, majd a hókotró a telephelyen felszereli az új fejet.
Aktorok	Takarító (játékos)
Forgatókönyv	1. A takarító meghívja az IntegratedMarket.buyItem(sp, newHead)-et. 2. A buyItem() hívásakor a Skeleton megkérdezi a felhasználót: 'Van elegendő pénze a játékosnak a vásárláshoz? (I/N)'. 'I' válasz esetén a tranzakció létrejön (objektum átadása, konstruktor hívása). 'N' válasz esetén a tranzakció megszakad, false értékkel tér vissza.. 4. A hókotró visszatér a telephelyre (SD-06 alapján). 5. HomeBase.swapHead(sp, newHead) meghívja a SnowPlow.changeHead(newHead)-et. 6. activeHead = newHead.

Use-case neve	SD-13: Üzemanyag vásárlás és töltés
Rövid leírás	A takarító üzemanyagot vásárol, majd a telephelyen a hókotró aktív fejét feltölti.
Aktorok	Takarító (játékos)
Forgatókönyv	1. A takarító meghívja az IntegratedMarket.buyItem(sp, fuelItem)-et. 2. A buyItem() hívásakor a Skeleton megkérdezi a felhasználót: 'Van elegendő pénze a játékosnak a vásárláshoz? (I/N)'. 'I' válasz esetén a tranzakció létrejön (objektum átadása, konstruktor hívása). 'N' válasz esetén a tranzakció megszakad, false értékkel tér vissza.. 4. A hókotró visszatér a telephelyre (SD-06 alapján). 5. HomeBase.refuelSnowPlow(sp, amount) hívódik. 6. SaltHead.refillFuel(amount) vagy DragonHead.refillFuel(amount): fuelAmount += amount.

Use-case neve	SD-14: Hókotró vásárlása
Rövid leírás	A takarító játékos új hókotrót vásárol, amelyet a telephely befogad.
Aktorok	Takarító (játékos)
Forgatókönyv	1. A takarító meghívja az IntegratedMarket.buyItem(existingSp, newSp)-et. 2. A buyItem() hívásakor a Skeleton megkérdezi a felhasználót: 'Van elegendő pénze a játékosnak a vásárláshoz? (I/N)'. 'I' válasz esetén a tranzakció létrejön (objektum átadása, konstruktor hívása). 'N' válasz esetén a tranzakció megszakad, false értékkel tér vissza..

Use-case neve	SD-15: Autó elakadása és átsorolás
Rövid leírás	Ha az autó célsávja le van zárva, a szomszédos szabad sávba sorol át.
Aktorok	Rendszer (GameLogic, Car)
Forgatókönyv	1. A Car.move() kiszámítja az útvonalat: nextField = l2. 2. Az isBlocked() hívásakor az l2 sávon a Skeleton megkérdezi: 'Le van zárva az l2 sáv? (I/N)'. A felhasználó 'I' választ ad a teszteset folytatásához, így a changeLane() hívódik. 3. Car.changeLane() hívódik. 4. lane.getNeighbors() visszaadja a szomszédos sávokat. 5. Az isBlocked() hívásakor a szomszéd sávon a Skeleton megkérdezi: 'Szabad a szomszédos sáv? (I/N)'. 'I' válasz esetén megtörténnek a remove(car) és accept(car) hívások az átsoroláshoz.

Use-case neve	SD-16: Játék vége és eredményhirdetés
Rövid leírás	Ha elérte a maximum körszámot, a GameLogic összesíti az eredményeket és kihirdeti a győztest.
Aktorok	Rendszer (GameLogic)
Forgatókönyv	1. Az advanceTurn() hívásakor a Skeleton megkérdezi: 'Elérte a körök száma a maximumot? (I/N)'. 'I' válasz esetén a rendszer meghívja az endGame() metódust. 2. GameLogic.endGame() meghívódik. 3. Minden Bus esetén getName() és completedRounds lekérdezése. 4. Minden SnowPlow esetén getName() és money lekérdezése. 5. Eredmények összesítése: - Legtöbb forduló = buszvezető győztes. - Legtöbb pénz = takarító győztes. 6. Az eredmény visszaadódik.

5.2 A szkeleton kezelői felületének terve, dialógusok

A szkeleton program egy kizárólag szöveges (konzolos) felülettel rendelkező alkalmazás, amely a szabványos bemenetet (stdin) és kimenetet (stdout) használja. Célja a tervezett analízis modell belső üzenetváltásainak (metódushívásainak) ellenőrzése interaktív teszteseteken keresztül.

5.2.1 Főmenü és teszteset kiválasztása

A program indításakor a felhasználót egy főmenü fogadja, ahol a megtervezett Use-case-ek (teszt-szekvenciák) közül választhat egy sorszám megadásával. Minden menüpont kiválasztása lefuttatja a hozzá tartozó inicializáló kódsort (kommunikációs diagram alapján), majd elindítja magát a szekvenciát.

5.2.2 A kimenet formátuma (Szekvenciák naplózása)

Annak érdekében, hogy a konzolos kimenet egyértelműen összevethető legyen az analízis modell szekvenciadiagramjaival, a rendszer egy központi Skeleton (vagy Logger) osztályon keresztül naplózza a metódushívásokat és a visszatéréseket.

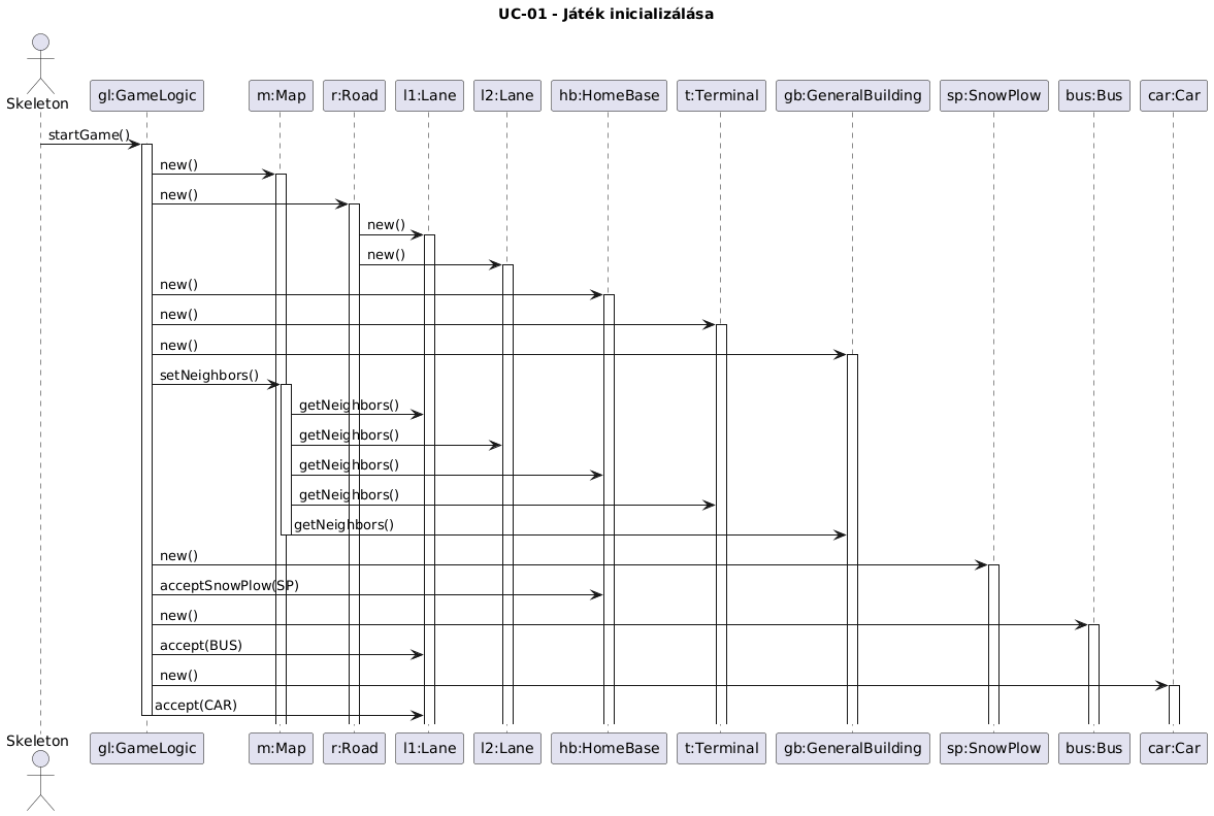
- **Behúzás (Indentation):** Minden egymásba ágyazott metódushívás egy tabulátorral (vagy adott számú szóközzel) beljebb kezdődik, reprezentálva a hívási verem (call stack) mélységét és a szekvenciadiagramok életvonalain lévő aktivációs dobozokat.
- **Hívás formátuma:** [Hívó_Objektum] -> [Hívott_Objektum].metódusNév(paraméterek)
- **Visszatérés formátuma:** <-- visszatérési_érték (az aktuális behúzási szintnek megfelelően).

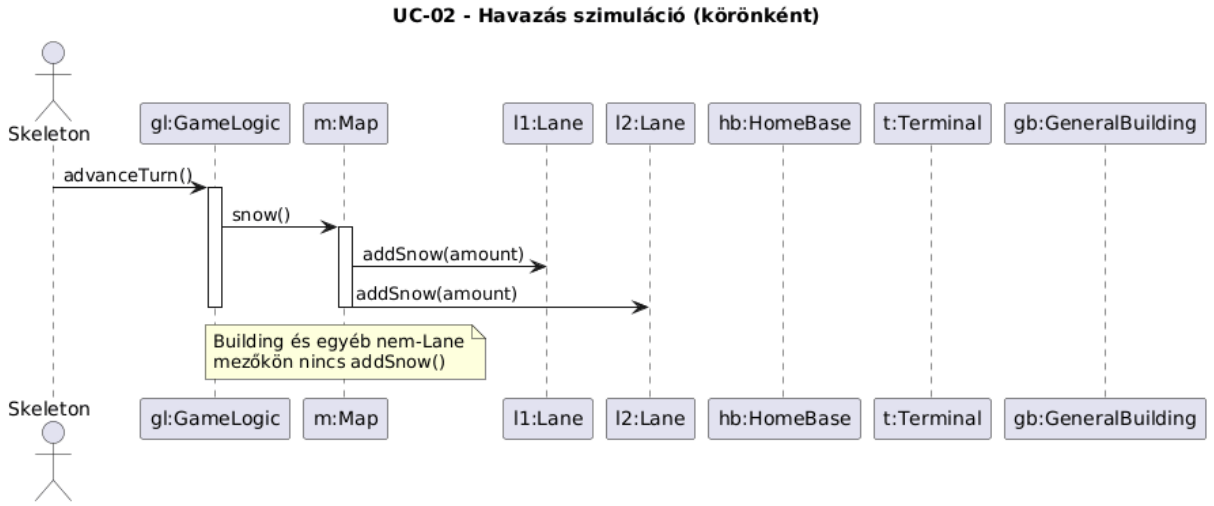
5.2.3 Dialógusok és felhasználói beavatkozás (I/N kérdések)

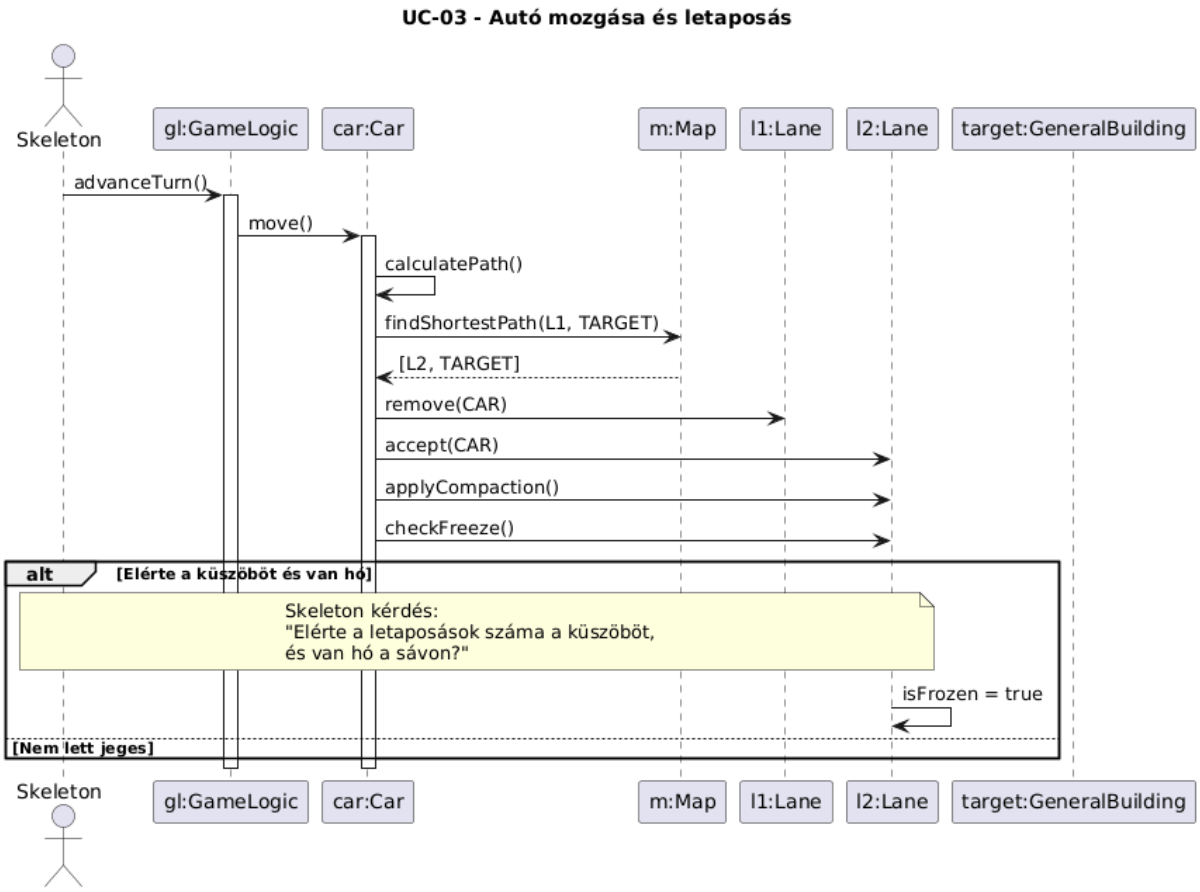
Mivel a szkeleton objektumai nem tárolnak belső állapotot (pl. pontos üzemanyagszintet, hóvastagságot) és nincsenek benne véletlenszerűségek, a feltételes elágazásoknál a rendszernek a tesztelőt (felhasználót) kell megkérdeznie. Ezt a központi Skeleton osztály végzi el.

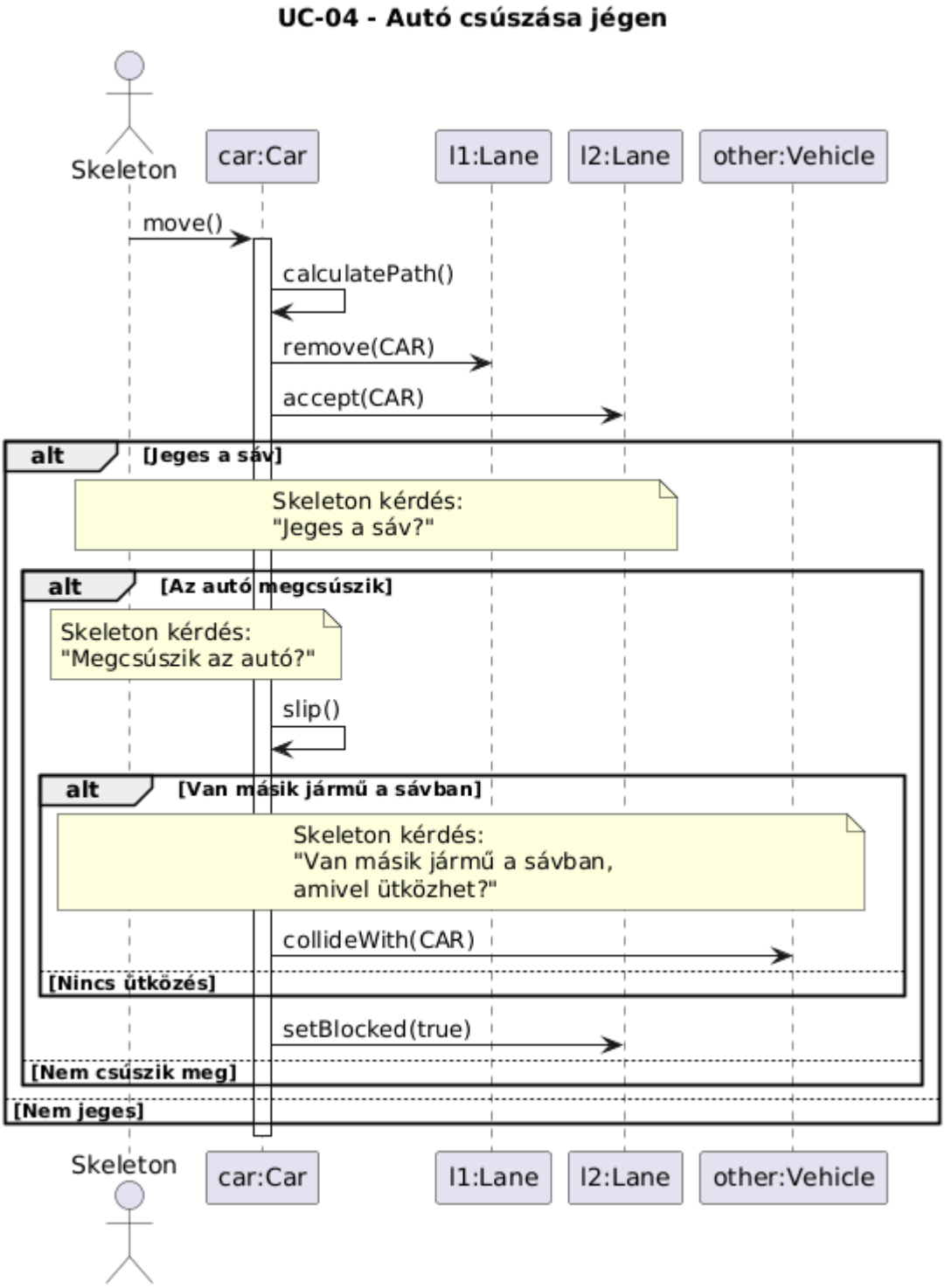
- A kérdések formátuma egységesen: ? [Kérdés szövege] (I/N):
- A felhasználó I (Igen) vagy N (Nem) karakterek beírásával irányítja a teszt futását.

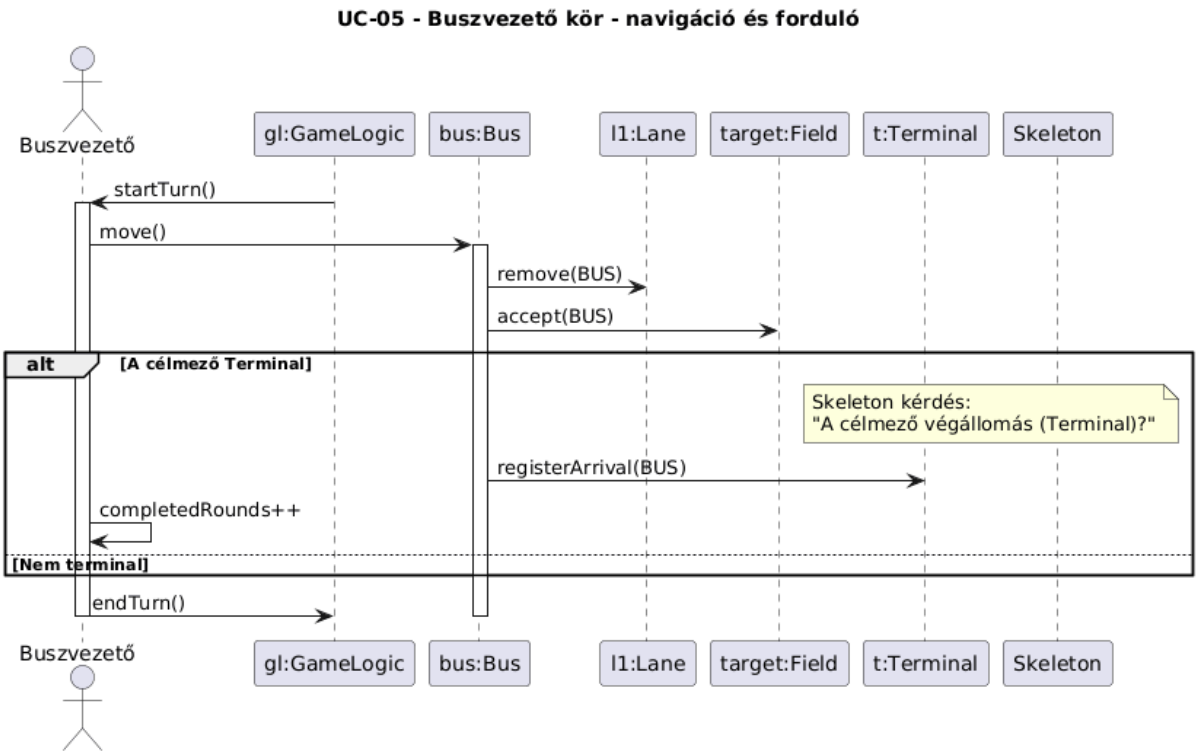
5.3 Szekvencia diagramok

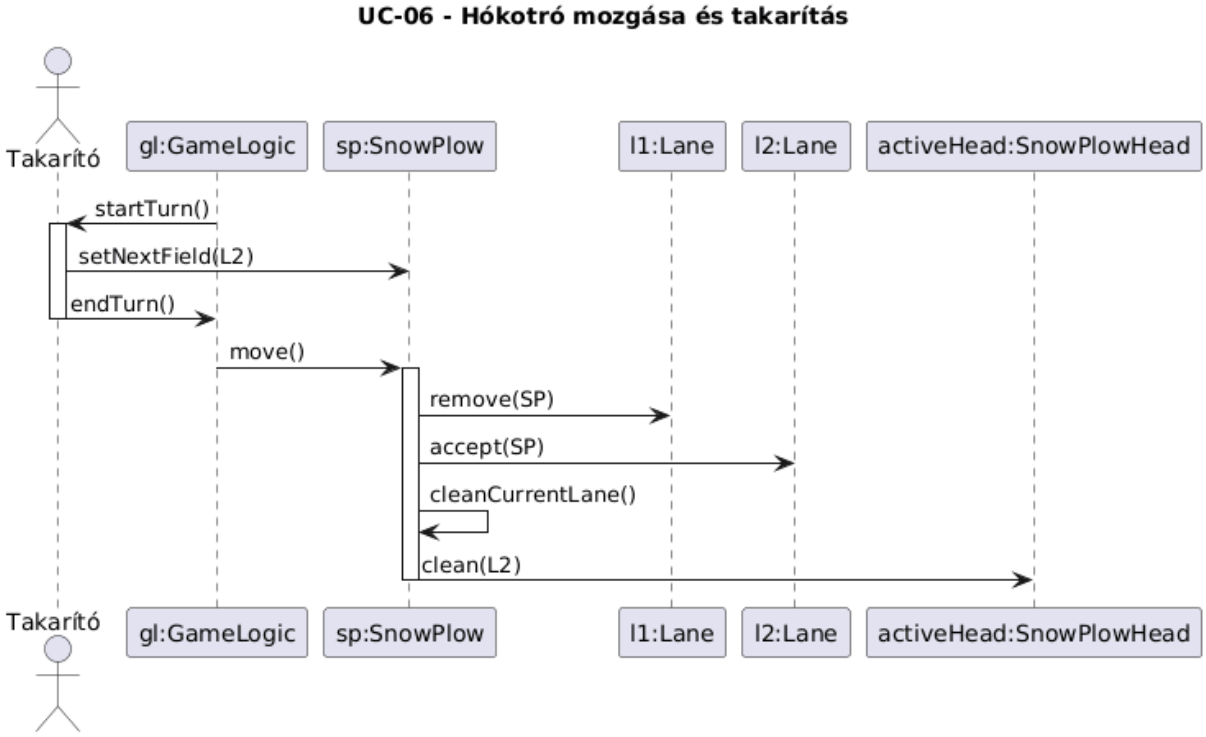


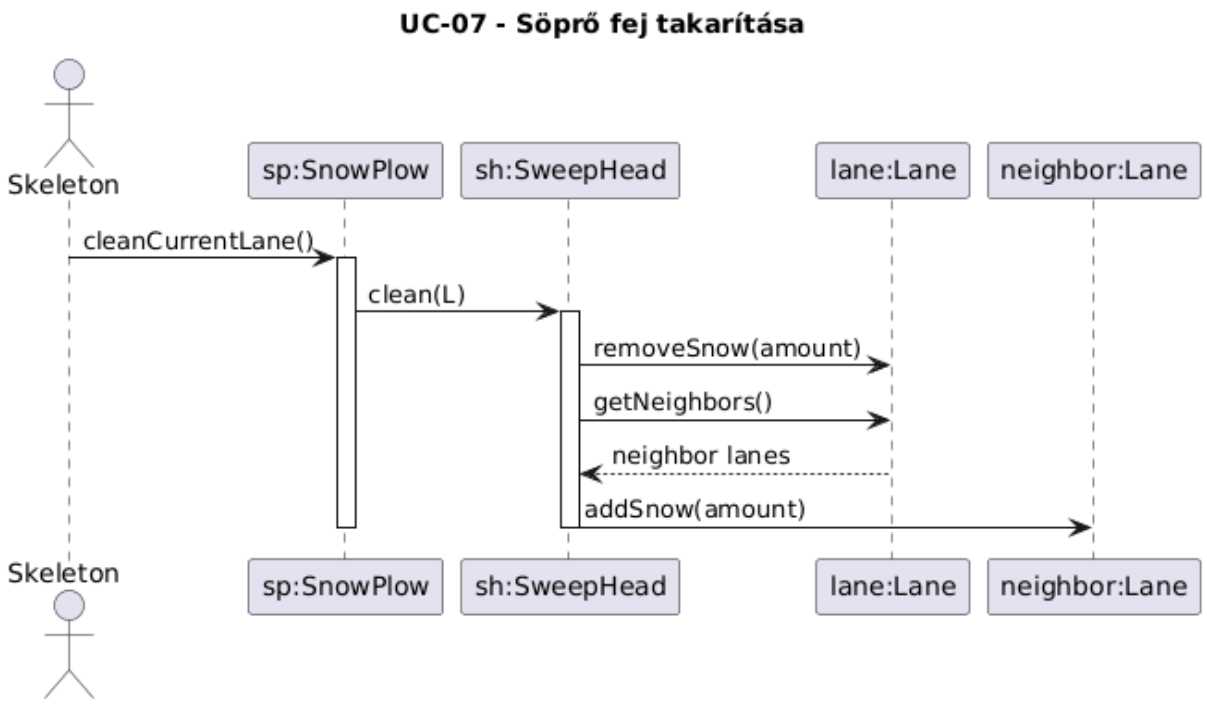


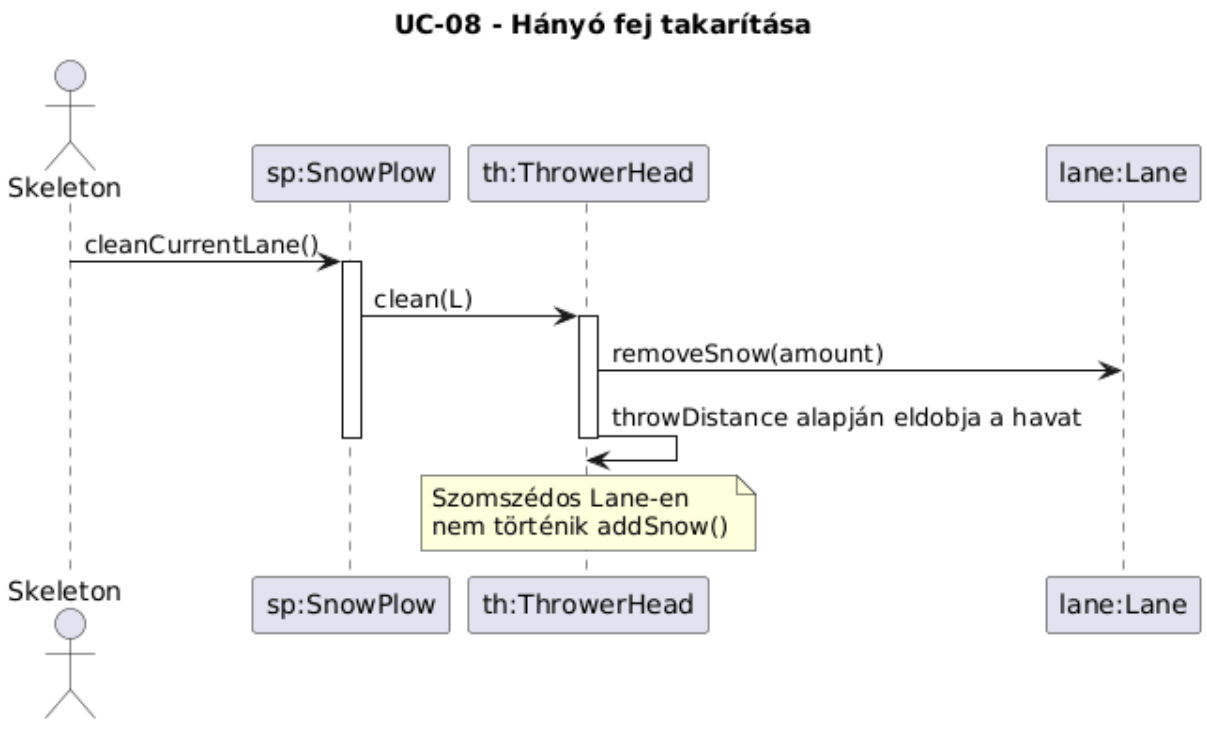

UC-03 - Autó mozgása és letaposás
Skeleton
gl:GameLogic
car:Car
m:Map
l1:Lane
l2:Lane
target:GeneralBuilding
advanceTurn()
move()
calculatePath()
findShortestPath(L1, TARGET)
[L2, TARGET]
remove(CAR)
accept(CAR)
applyCompaction()
checkFreeze()
alt
[Elérte a küszöböt és van hó]
Skeleton kérdés:
"Elérte a letaposások száma a küszöböt,
és van hó a sávon?"
isFrozen = true
[Nem lett jeges]
Skeleton
gl:GameLogic
car:Car
m:Map
l1:Lane
l2:Lane
target:GeneralBuilding

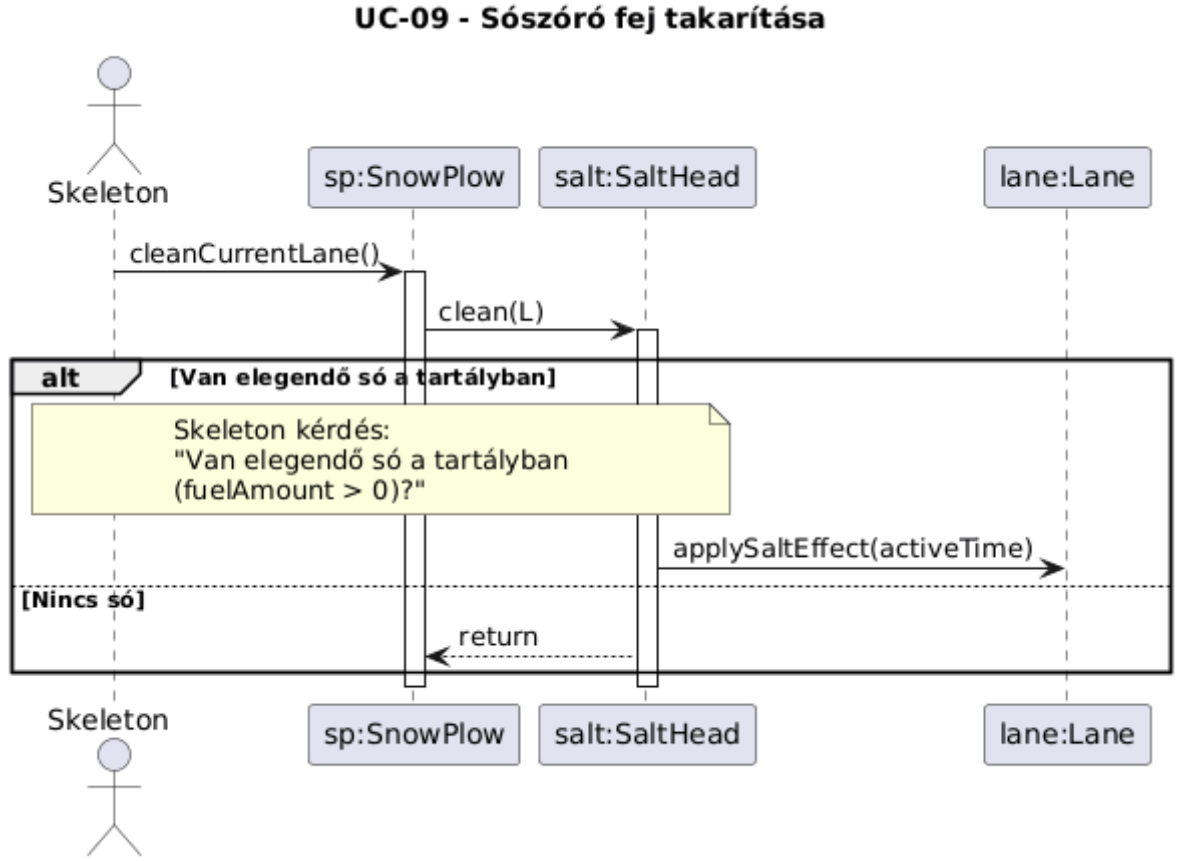

UC-04 - Autó csúszása jégen
Skeleton
car:Car
l1:Lane
l2:Lane
other:Vehicle
move()
calculatePath()
remove(CAR)
accept(CAR)
alt
[Jeges a sáv]
Skeleton kérdés:
"Jeges a sáv?"
alt
[Az autó megcsúszik]
Skeleton kérdés:
"Megcsúszik az autó?"
slip()
alt
[Van másik jármű a sávban]
Skeleton kérdés:
"Van másik jármű a sávban,
amivel ütközhet?"
collideWith(CAR)
[Nincs ütközés]
setBlocked(true)
[Nem csúszik meg]
[Nem jeges]
Skeleton
car:Car
l1:Lane
l2:Lane
other:Vehicle

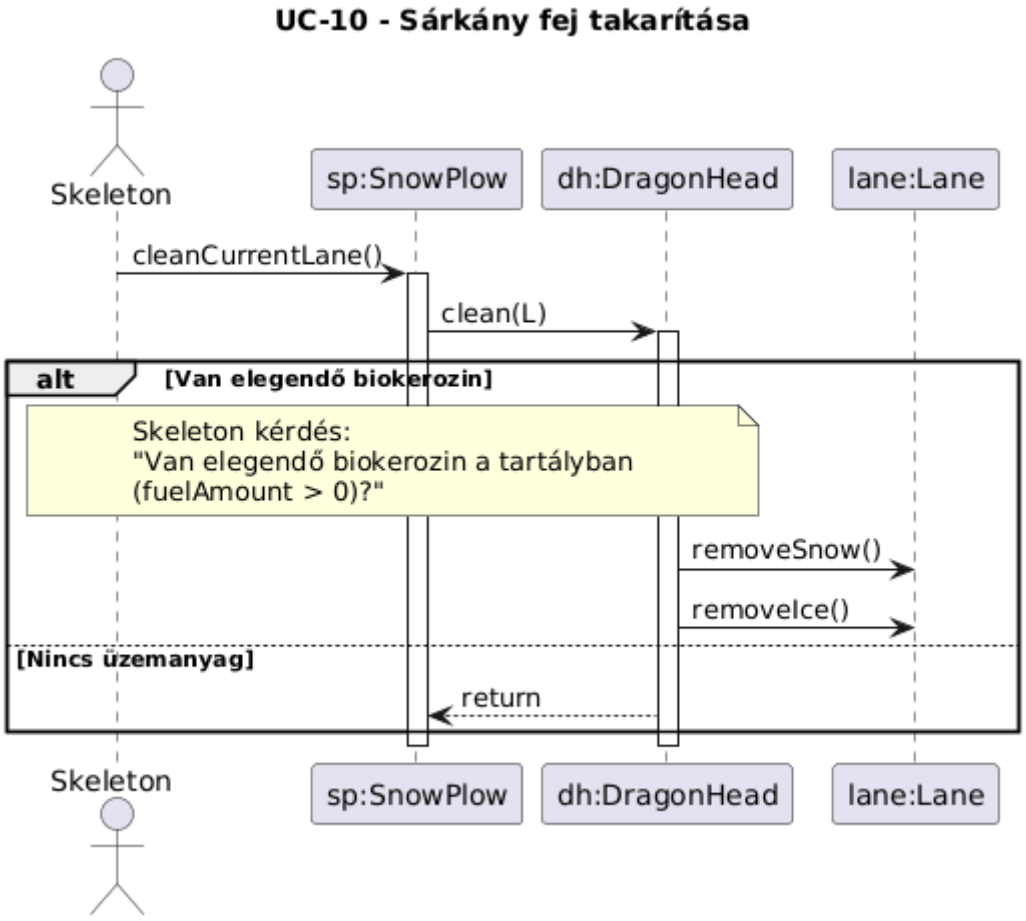

UC-05 - Buszvezető kör - navigáció és forduló
Buszvezető
gl:GameLogic
bus:Bus
l1:Lane
target:Field
t:Terminal
Skeleton
startTurn()
move()
remove(BUS)
accept(BUS)
alt
[A célmező Terminal]
Skeleton kérdés:
"A célmező végállomás (Terminal)?"
registerArrival(BUS)
completedRounds++
[Nem terminal]
endTurn()

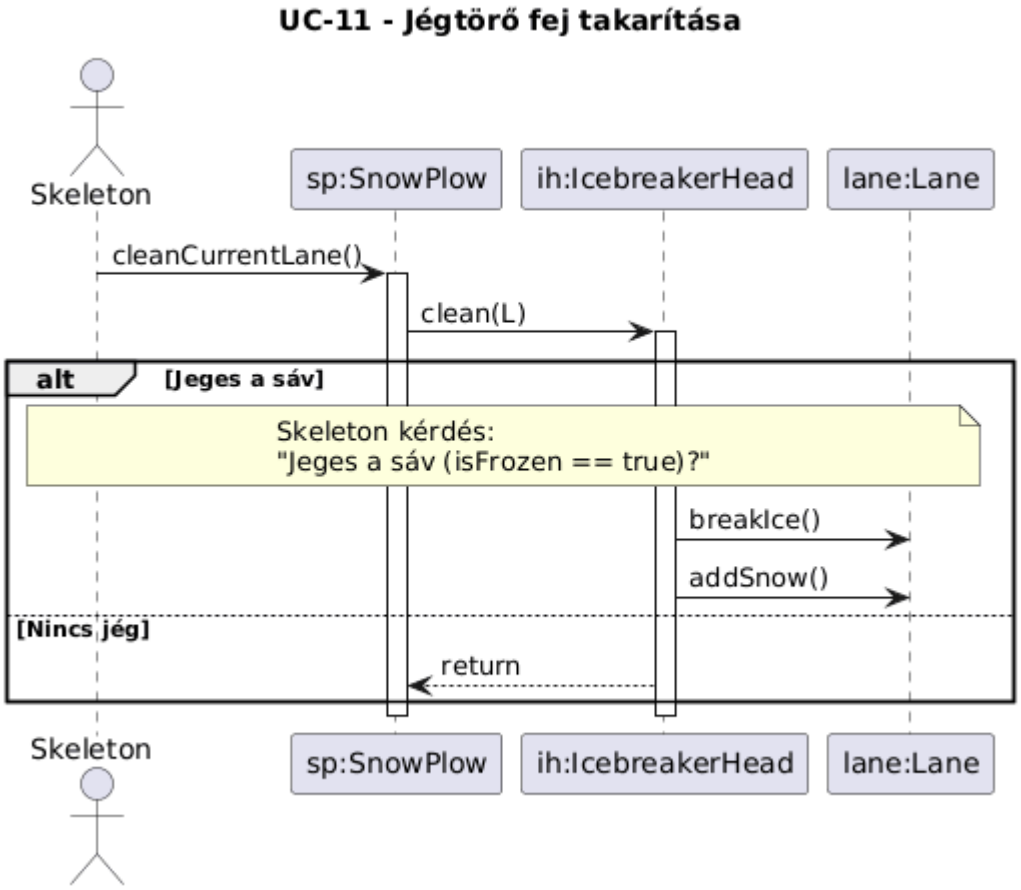

UC-06 - Hókotró mozgása és takarítás
Takarító
gl:GameLogic
sp:SnowPlow
l1:Lane
l2:Lane
activeHead:SnowPlowHead
startTurn()
setNextField(L2)
endTurn()
move()
remove(SP)
accept(SP)
cleanCurrentLane()
clean(L2)

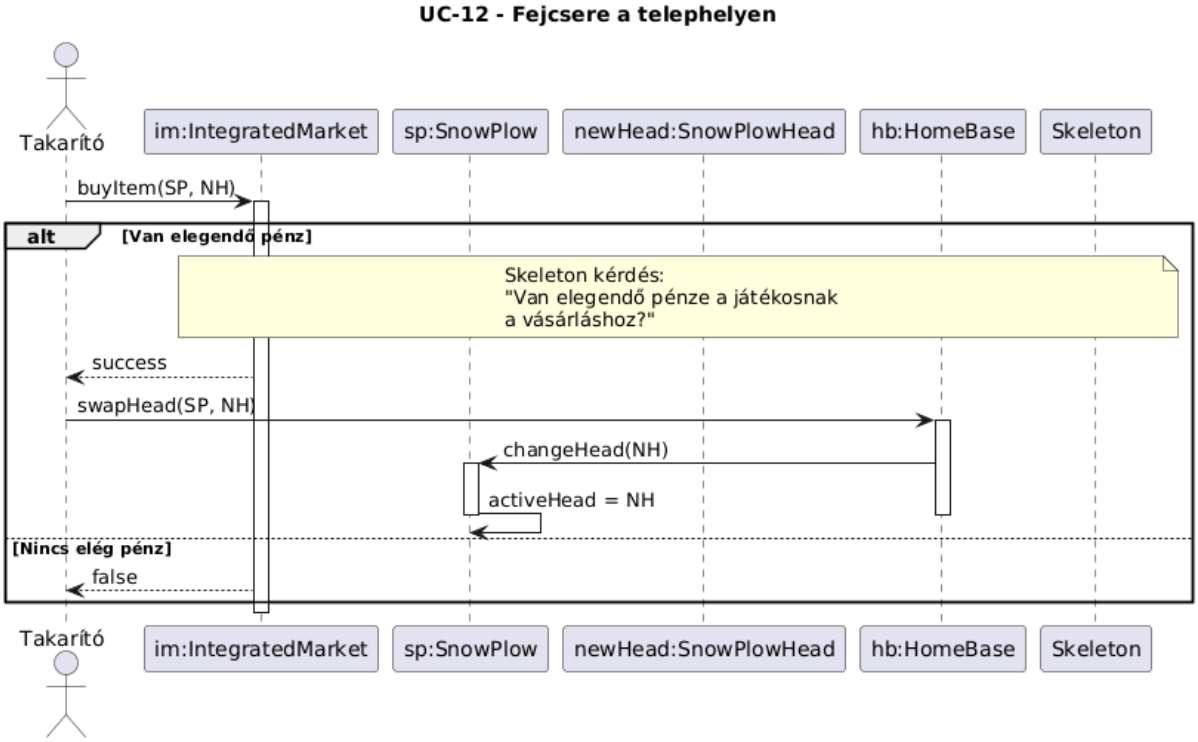

UC-07 - Söprő fej takarítása
Skeleton
sp:SnowPlow
sh:SweepHead
lane:Lane
neighbor:Lane
cleanCurrentLane()
clean(L)
removeSnow(amount)
getNeighbors()
neighbor lanes
addSnow(amount)
Skeleton
sp:SnowPlow
sh:SweepHead
lane:Lane
neighbor:Lane

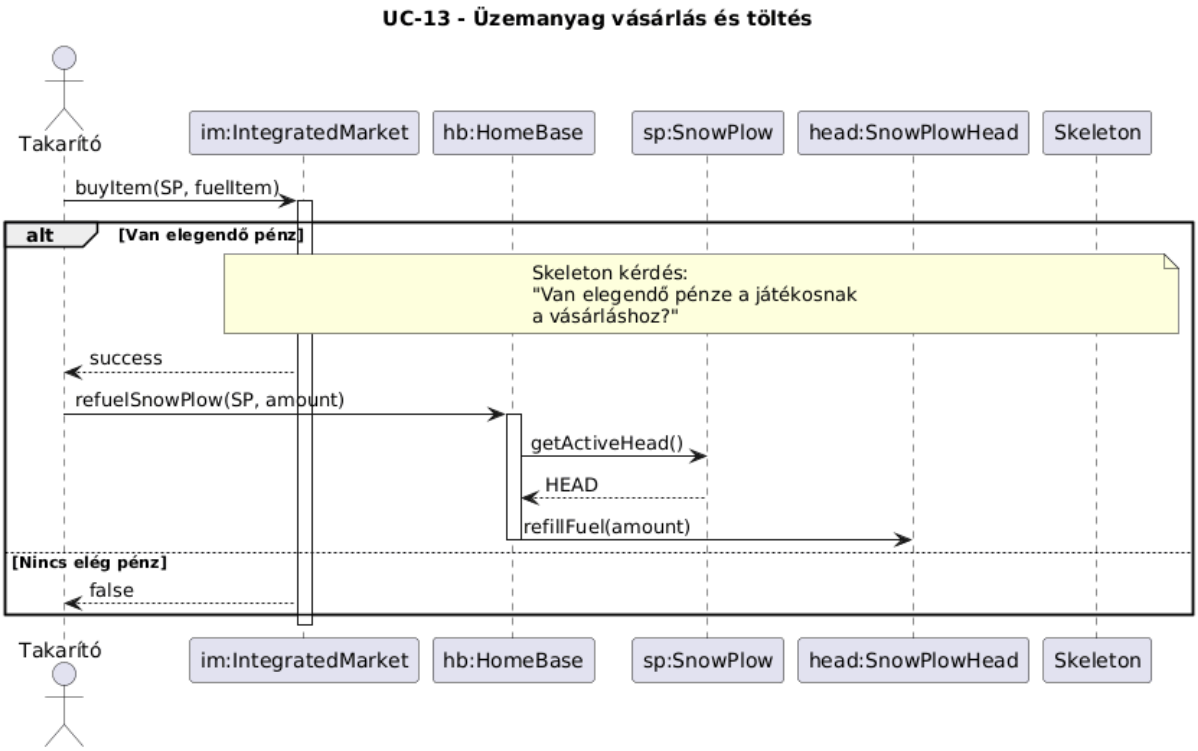

UC-08 - Hányó fej takarítása
Skeleton
sp:SnowPlow
th:ThrowerHead
lane:Lane
cleanCurrentLane()
clean(L)
removeSnow(amount)
throwDistance alapján eldobja a havat
Szomszédos Lane-en
nem történik addSnow()
Skeleton
sp:SnowPlow
th:ThrowerHead
lane:Lane

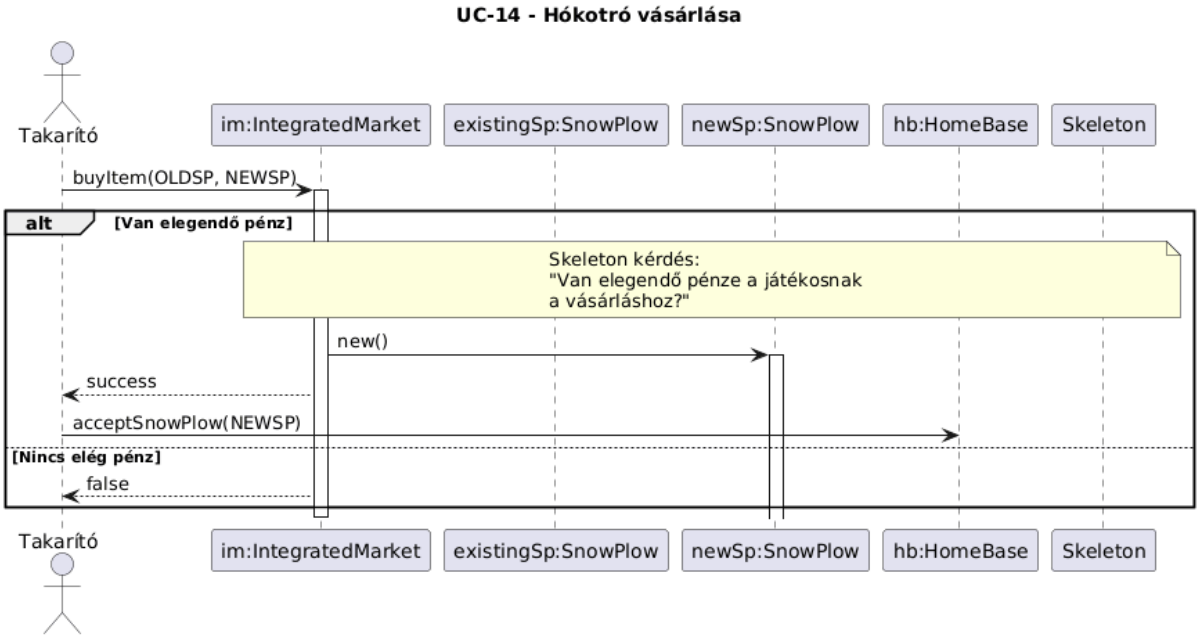

UC-09 - Sószóró fej takarítása
Skeleton
sp:SnowPlow
salt:SaltHead
lane:Lane
cleanCurrentLane()
clean(L)
alt
[Van elegendő só a tartályban]
Skeleton kérdés:
"Van elegendő só a tartályban
(fuelAmount > 0)?"
applySaltEffect(activeTime)
[Nincs só]
return
Skeleton
sp:SnowPlow
salt:SaltHead
lane:Lane

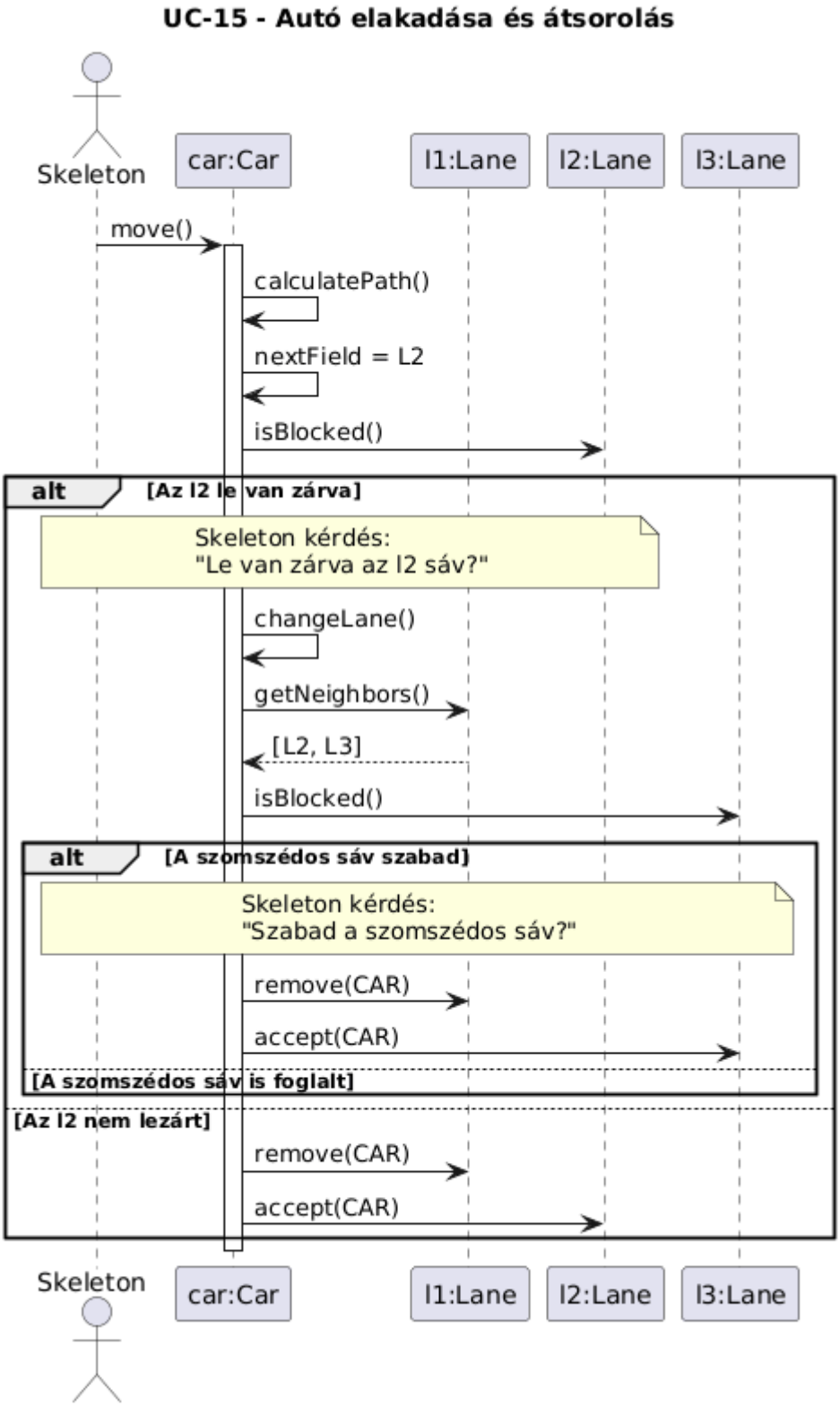

UC-10 - Sárkány fej takarítása
Skeleton
sp:SnowPlow
dh:DragonHead
lane:Lane
cleanCurrentLane()
clean(L)
alt
[Van elegendő biokerozin]
Skeleton kérdés:
"Van elegendő biokerozin a tartályban
(fuelAmount > 0)?"
removeSnow()
removeIce()
[Nincs üzemanyag]
return
Skeleton
sp:SnowPlow
dh:DragonHead
lane:Lane

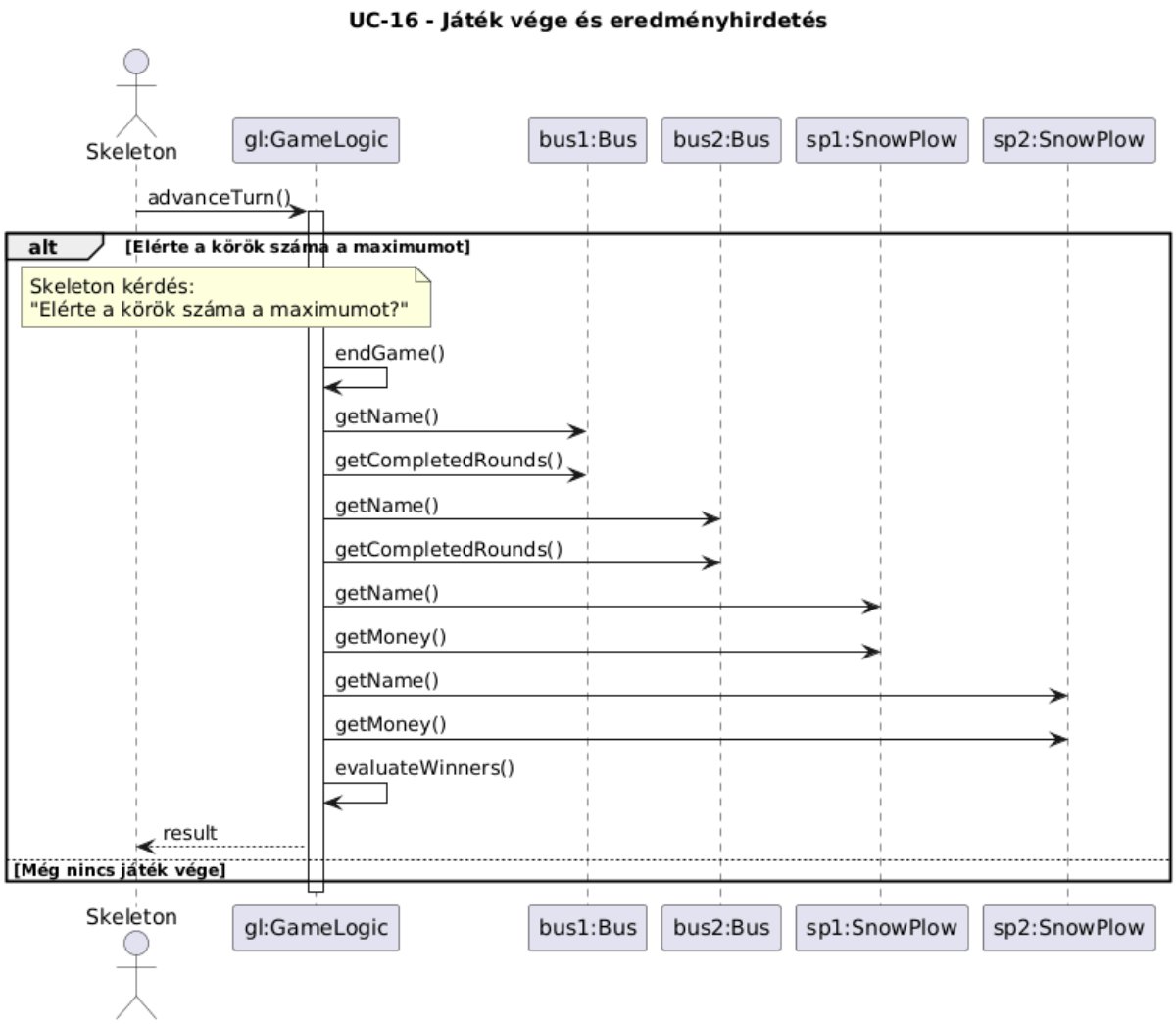

UC-11 - Jégtörő fej takarítása
Skeleton
sp:SnowPlow
ih:IcebreakerHead
lane:Lane
cleanCurrentLane()
clean(L)
alt
[Jeges a sáv]
Skeleton kérdés:
"Jeges a sáv (isFrozen == true)?"
breakIce()
addSnow()
[Nincs jég]
return
Skeleton
sp:SnowPlow
ih:IcebreakerHead
lane:Lane


UC-12 - Fejcsere a telephelyen
Takarító
im:IntegratedMarket
sp:SnowPlow
newHead:SnowPlowHead
hb:HomeBase
Skeleton
buyItem(SP, NH)
alt
[Van elegendő pénz]
Skeleton kérdés:
"Van elegendő pénze a játékosnak
a vásárláshoz?"
success
swapHead(SP, NH)
changeHead(NH)
activeHead = NH
[Nincs elég pénz]
false
Takarító
im:IntegratedMarket
sp:SnowPlow
newHead:SnowPlowHead
hb:HomeBase
Skeleton

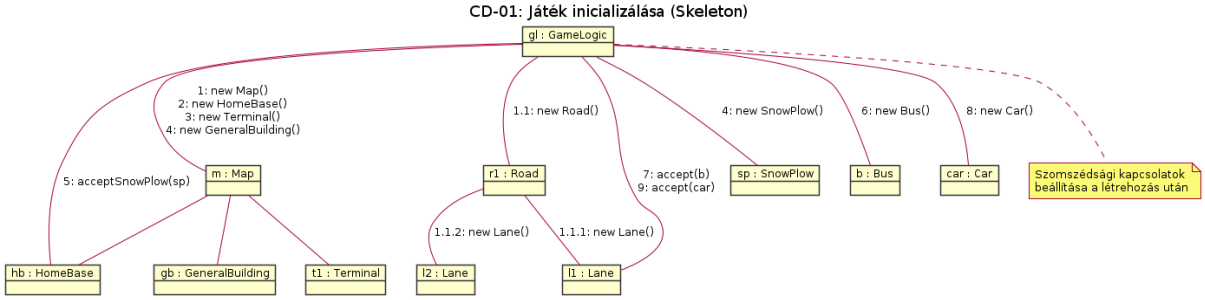

UC-13 - Üzemanyag vásárlás és töltés
Takarító
im:IntegratedMarket
hb:HomeBase
sp:SnowPlow
head:SnowPlowHead
Skeleton
buyItem(SP, fuelItem)
alt
[Van elegendő pénz]
Skeleton kérdés:
"Van elegendő pénze a játékosnak
a vásárláshoz?"
success
refuelSnowPlow(SP, amount)
getActiveHead()
HEAD
refillFuel(amount)
[Nincs elég pénz]
false
Takarító
im:IntegratedMarket
hb:HomeBase
sp:SnowPlow
head:SnowPlowHead
Skeleton

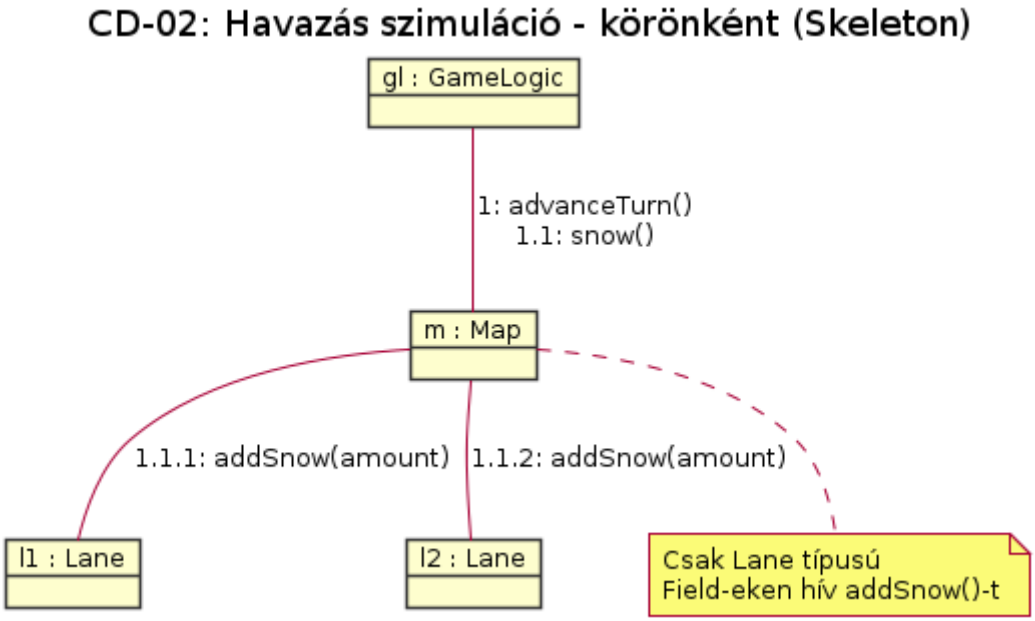

UC-14 - Hókotró vásárlása
Takarító
im:IntegratedMarket
existingSp:SnowPlow
newSp:SnowPlow
hb:HomeBase
Skeleton
buyItem(OLDSP, NEWSP)
alt
[Van elegendő pénz]
Skeleton kérdés:
"Van elegendő pénze a játékosnak
a vásárláshoz?"
new()
success
acceptSnowPlow(NEWSP)
[Nincs elég pénz]
false
Takarító
im:IntegratedMarket
existingSp:SnowPlow
newSp:SnowPlow
hb:HomeBase
Skeleton

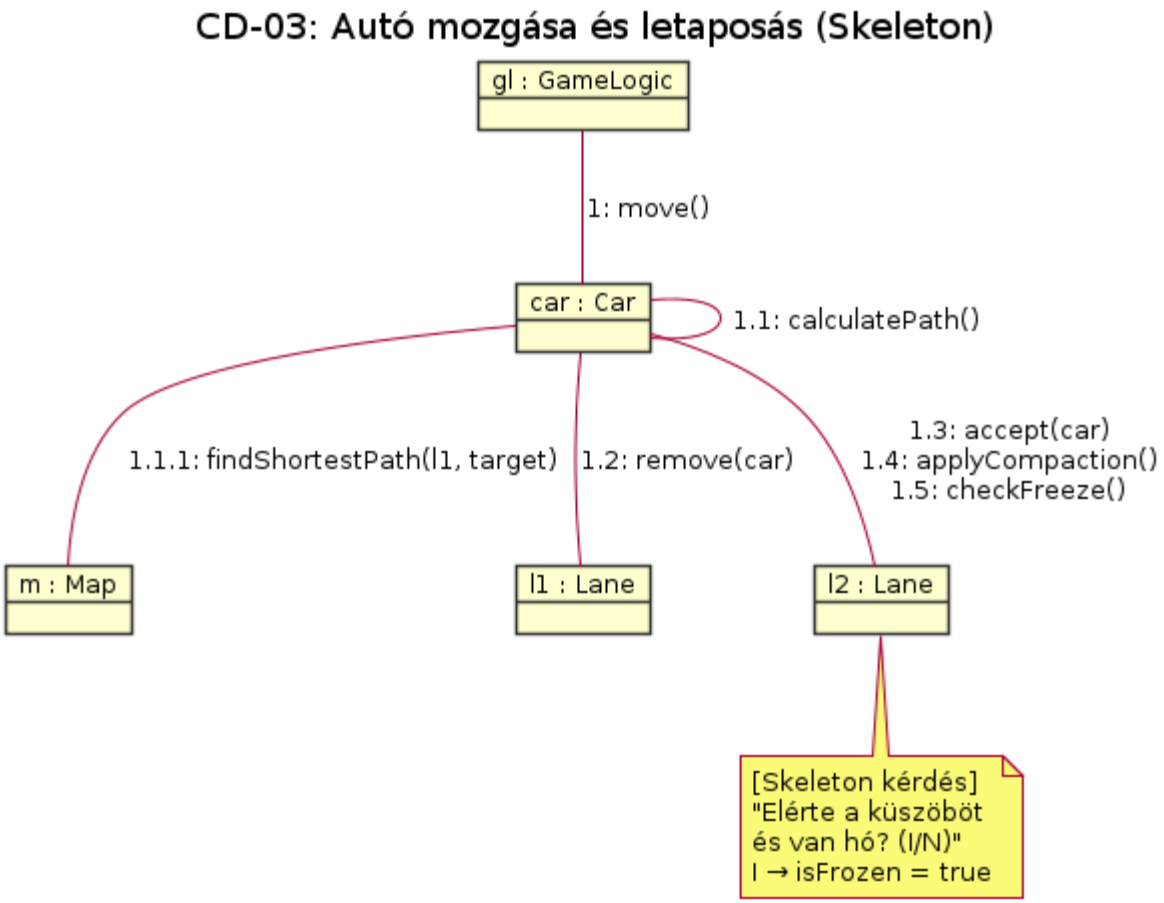

UC-15 - Autó elakadása és átsorolás
Skeleton
car:Car
l1:Lane
l2:Lane
l3:Lane
move()
calculatePath()
nextField = L2
isBlocked()
alt
[Az l2 le van zárva]
Skeleton kérdés:
"Le van zárva az l2 sáv?"
changeLane()
getNeighbors()
[L2, L3]
isBlocked()
alt
[A szomszédos sáv szabad]
Skeleton kérdés:
"Szabad a szomszédos sáv?"
remove(CAR)
accept(CAR)
[A szomszédos sáv is foglalt]
[Az l2 nem lezárt]
remove(CAR)
accept(CAR)
Skeleton
car:Car
l1:Lane
l2:Lane
l3:Lane

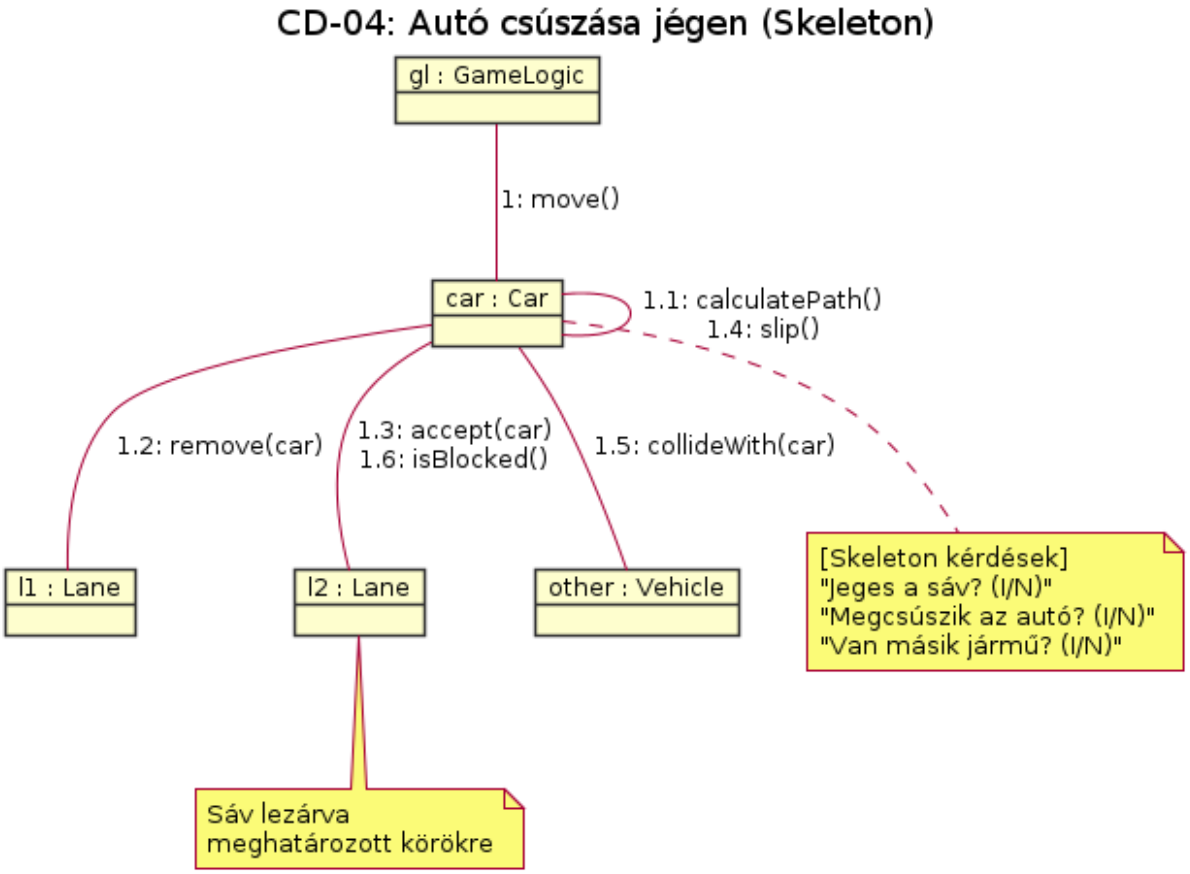

UC-16 - Játék vége és eredményhirdetés
Skeleton
gl:GameLogic
bus1:Bus
bus2:Bus
sp1:SnowPlow
sp2:SnowPlow
advanceTurn()
alt
[Elérte a körök száma a maximumot]
Skeleton kérdés:
"Elérte a körök száma a maximumot?"
endGame()
getName()
getCompletedRounds()
getName()
getCompletedRounds()
getName()
getMoney()
getName()
getMoney()
evaluateWinners()
result
[Még nincs játék vége]
Skeleton
gl:GameLogic
bus1:Bus
bus2:Bus
sp1:SnowPlow
sp2:SnowPlow

5.4 Kommunikációs diagramok

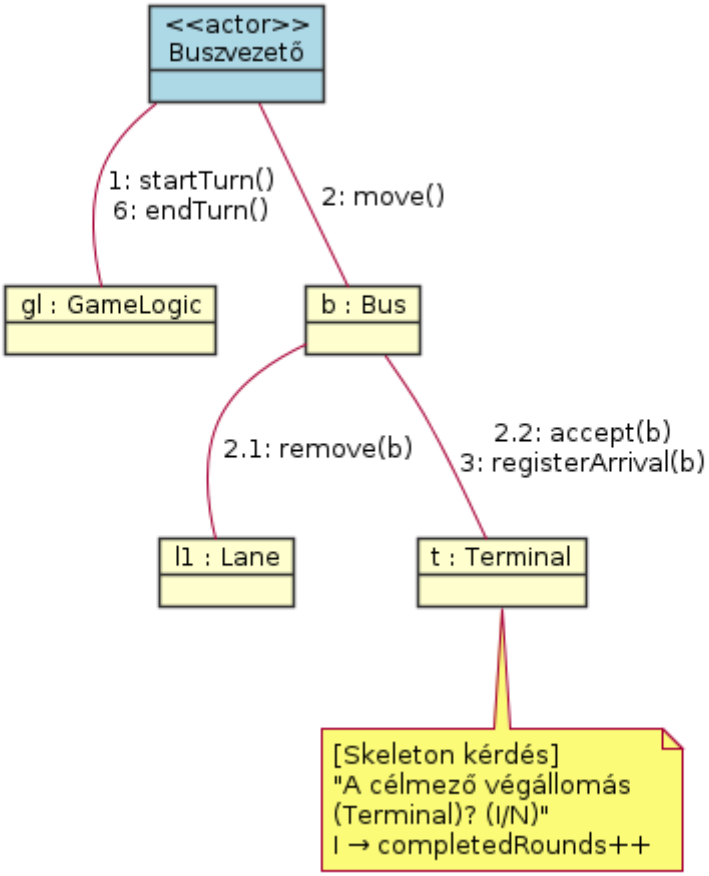




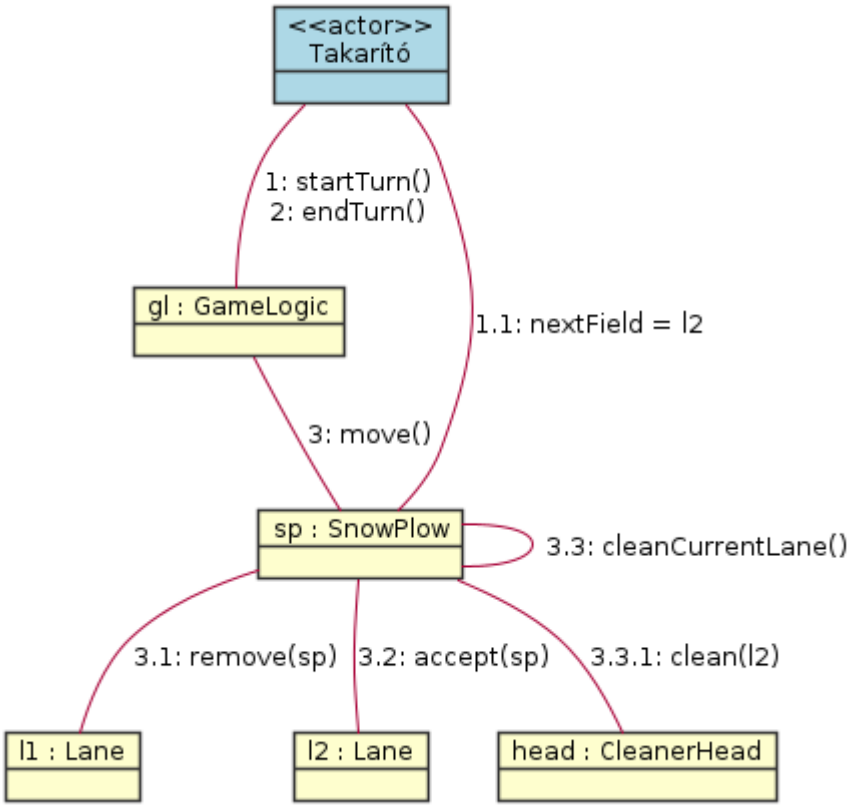

CD-03: Autó mozgása és letaposás (Skeleton)
gl : GameLogic
1: move()
car : Car
1.1: calculatePath()
1.1.1: findShortestPath(l1, target)
1.2: remove(car)
1.3: accept(car)
1.4: applyCompaction()
1.5: checkFreeze()
m : Map
l1 : Lane
l2 : Lane
[Skeleton kérdés]
"Elérte a küszöböt
és van hó? (I/N)"
I → isFrozen = true

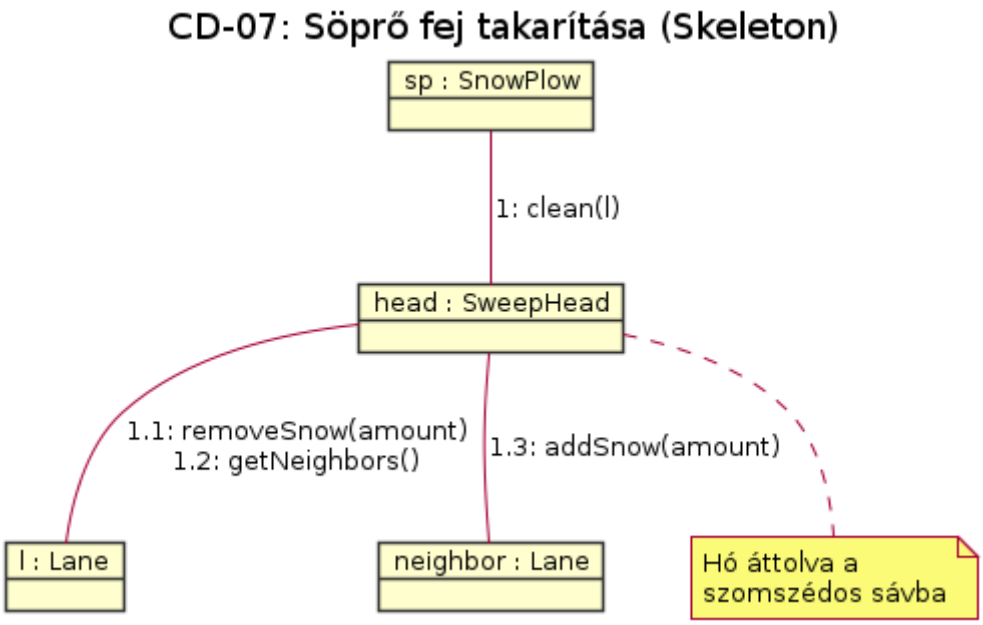

CD-04: Autó csúszása jégen (Skeleton)
gl : GameLogic
1: move()
car : Car
1.1: calculatePath()
1.4: slip()
1.2: remove(car)
1.3: accept(car)
1.6: isBlocked()
1.5: collideWith(car)
l1 : Lane
l2 : Lane
other : Vehicle
[Skeleton kérdések]
"Jeges a sáv? (I/N)"
"Megcsúszik az autó? (I/N)"
"Van másik jármű? (I/N)"
Sáv lezárva
meghatározott körökre

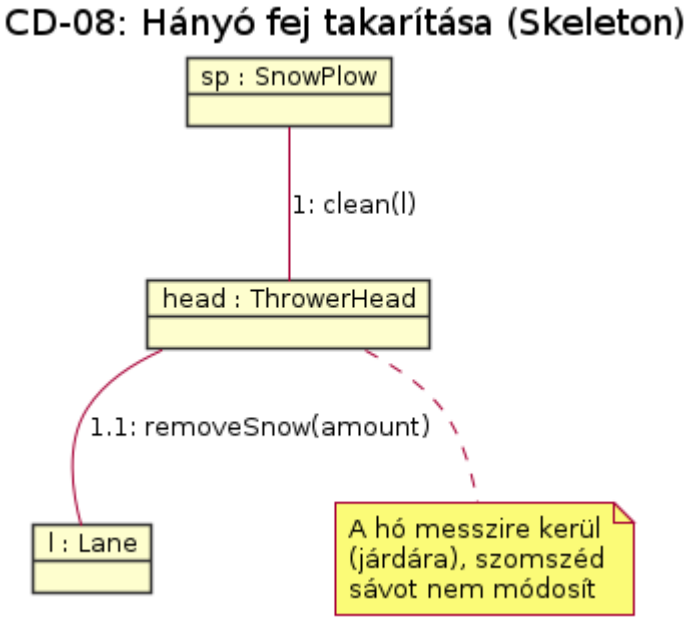
CD-05: Buszvezető kör - navigáció és forduló (Skeleton)



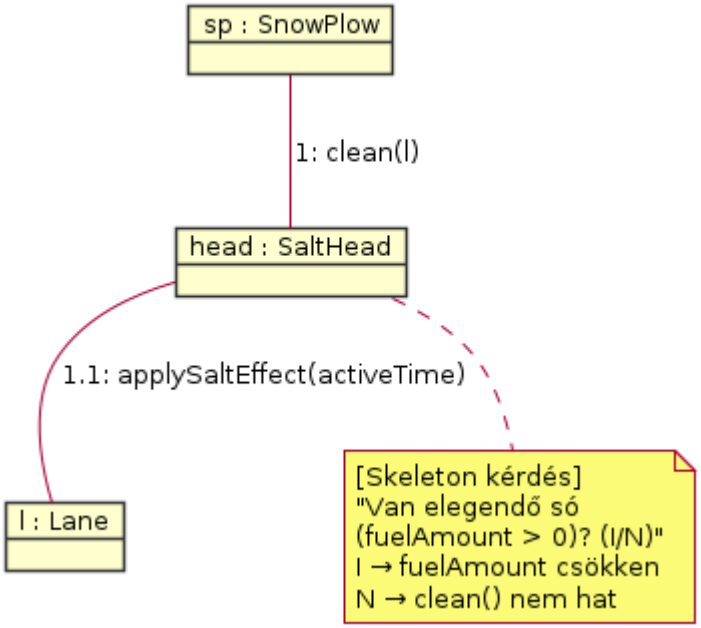
CD-06: Hókotró mozgása és takarítás (Skeleton)



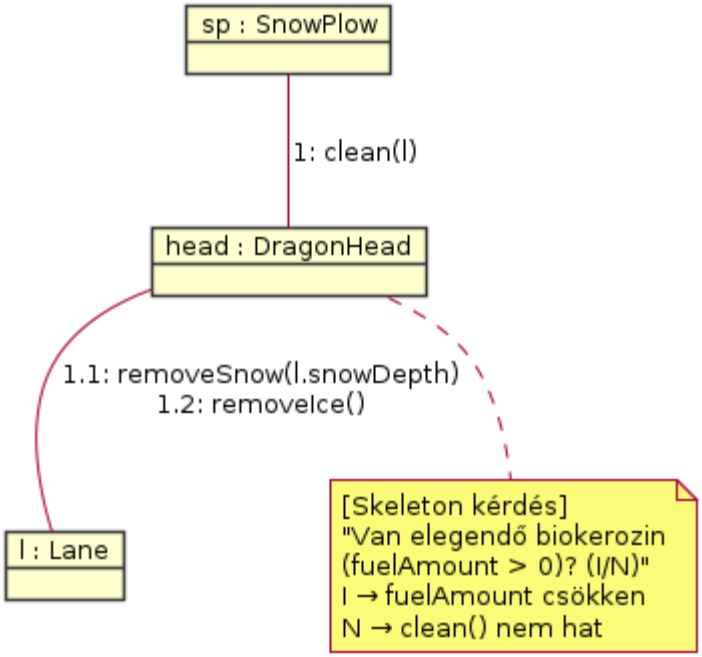

CD-07: Söprő fej takarítása (Skeleton)
sp : SnowPlow
1: clean(l)
head : SweepHead
1.1: removeSnow(amount)
1.2: getNeighbors()
1.3: addSnow(amount)
l : Lane
neighbor : Lane
Hó áttolva a szomszédos sávba


CD-08: Hányó fej takarítása (Skeleton)
sp : SnowPlow
1: clean(l)
head : ThrowerHead
1.1: removeSnow(amount)
l : Lane
A hó messzire kerül (járdára), szomszéd sávot nem módosít

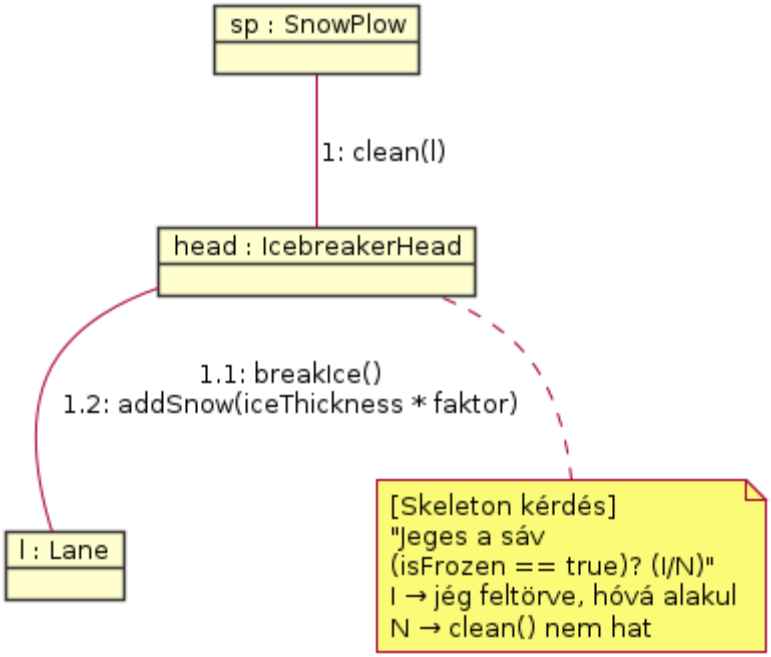
CD-09: Sószóró fej takarítása (Skeleton)



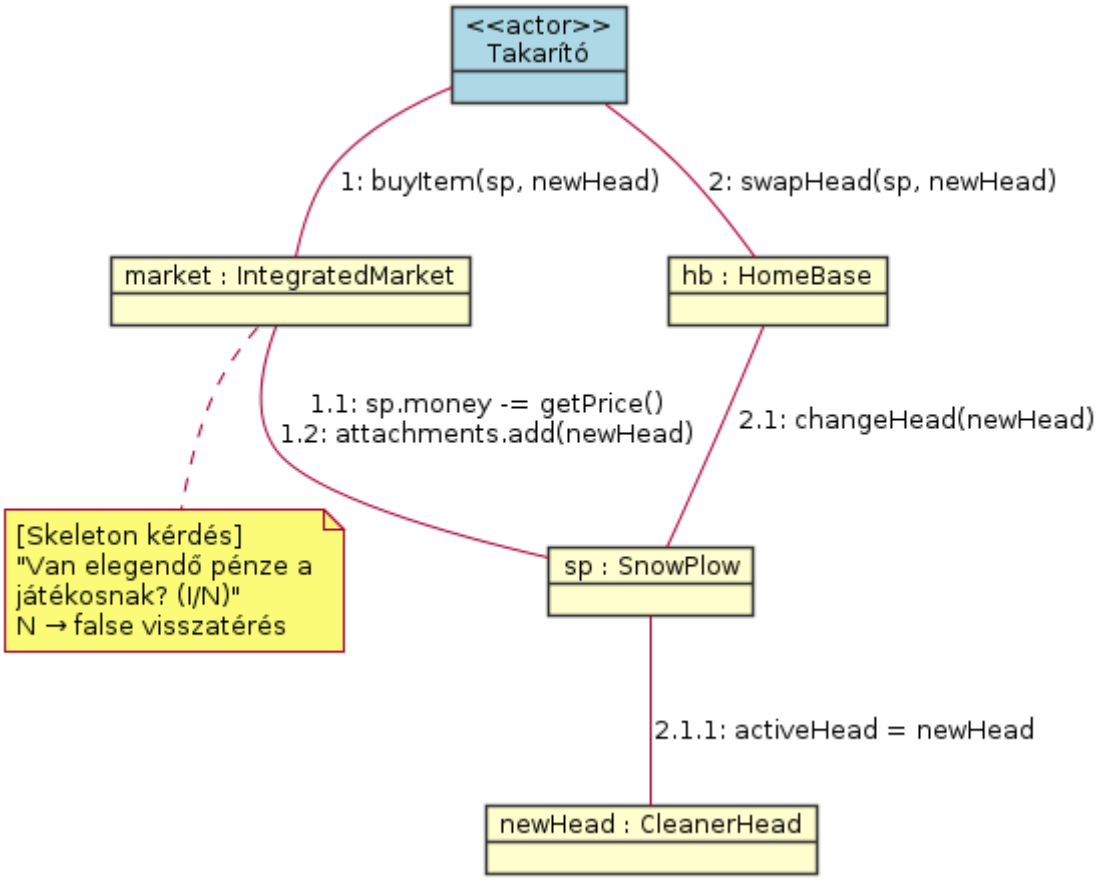
CD-10: Sárkány fej takarítása (Skeleton)



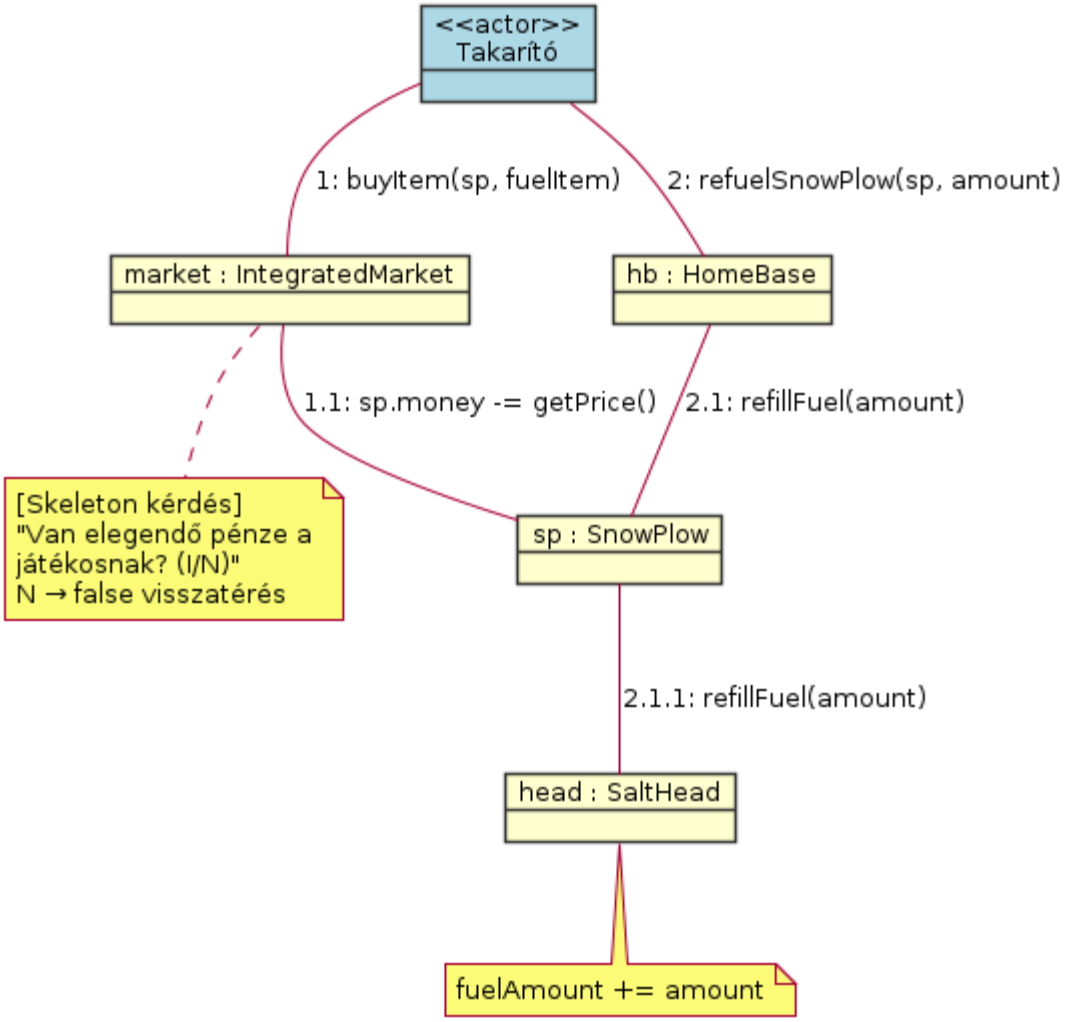
CD-11: Jégtörő fej takarítása (Skeleton)



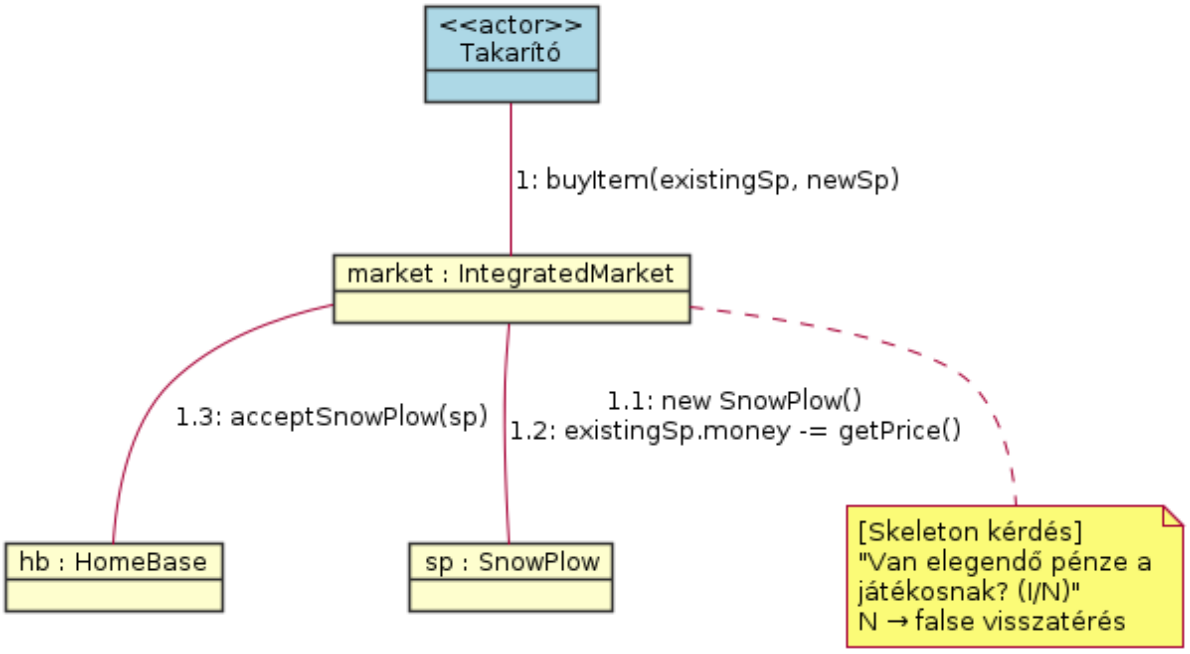
CD-12: Fejcsere a telephelyen (Skeleton)

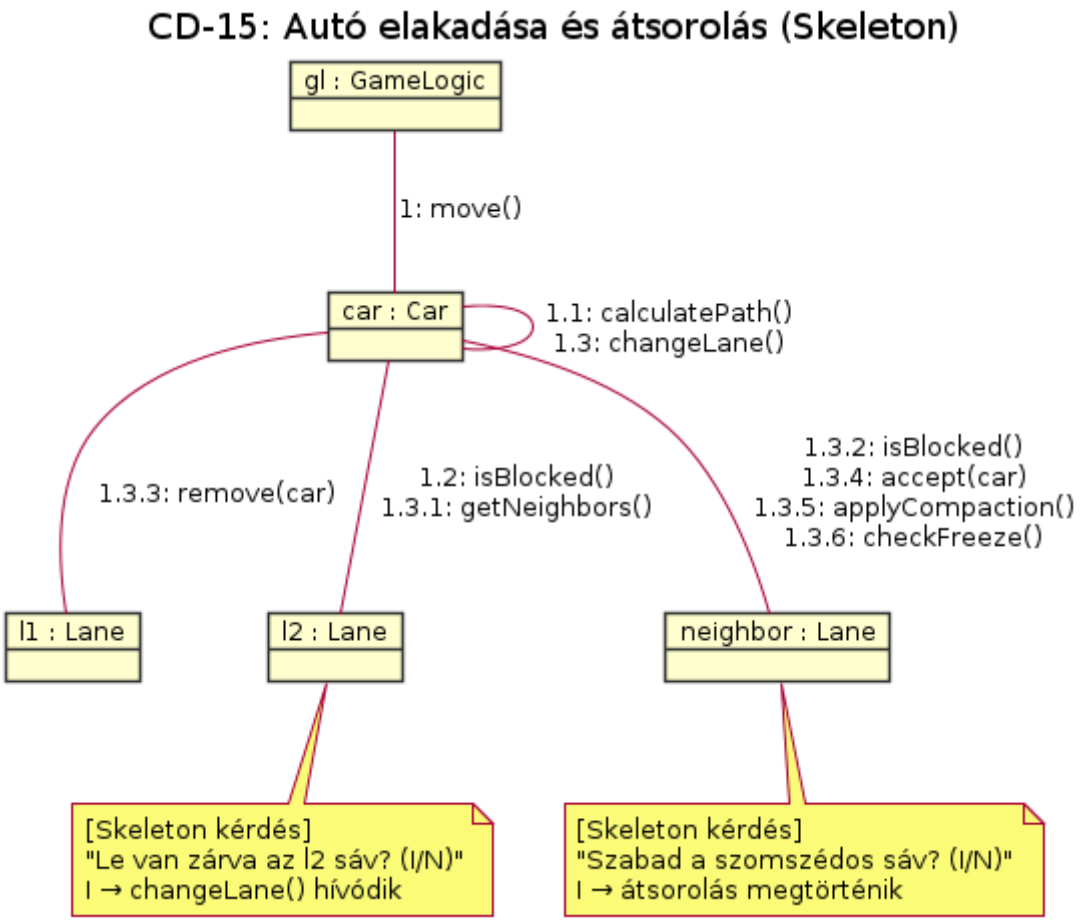


CD-13: Üzemanyag vásárlás és töltés (Skeleton)



CD-14: Hókotró vásárlása (Skeleton)




CD-15: Autó elakadása és átsorolás (Skeleton)
gl : GameLogic
1: move()
car : Car
1.1: calculatePath()
1.3: changeLane()
1.3.3: remove(car)
1.2: isBlocked()
1.3.1: getNeighbors()
1.3.2: isBlocked()
1.3.4: accept(car)
1.3.5: applyCompaction()
1.3.6: checkFreeze()
l1 : Lane
l2 : Lane
neighbor : Lane
[Skeleton kérdés]
"Le van zárva az l2 sáv? (I/N)"
I → changeLane() hívódik
[Skeleton kérdés]
"Szabad a szomszédos sáv? (I/N)"
I → átsorolás megtörténik

6.2 Napló

Kezdet	Időtartam	Résztvevők	Leírás
2026.03.19 19:00	2 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: Megállapodtunk mit készít el..
2026.03.20 20:00	4 óra	Kálmán	Tevékenység: Elkészítette a use-case leírásokat.
2026.03.21. 18:00	4 óra	Héja	Tevékenység: Elkészítette a use case diagrammokat.
2026.03.21. 02:00	4 óra	Papp	Tevékenység: Elkészítette a skeleton leírását.
2026.03.21. 10:00	4 óra	Grób	Tevékenység: Elkészítette a szekvencia diagrammokat
2026.03.22. 14:00	4 óra	Dobronyi	Tevékenység: Elkészítette a kommunikációs diagrammokat.
2026.03.22. 18:00	2 óra	Kálmán Héja Papp Grób Dobronyi	Értekezlet. Döntés: összeegyeztettük az egyénileg megcsinált részfeladatokat, és véglegesítettük a dokumentumot.